



Ostbayerische Technische Hochschule Regensburg

Fakultät Informatik und Mathematik

Studiengang Technische Informatik

Sommersemester 2026

Ausgewählte Projekte der Informatik

MotoSense Dokumentation

Autoren

Felix Kölling

felix.koelling@st.oth-regensburg.de

Inhaltsverzeichnis

1	Einleitung	4
1.1	Projektbeschreibung und Zielsetzung	4
1.2	Versuchsfahrzeug	4
1.3	Abgrenzung zum Projektvorschlag	4
2	Systemarchitektur	5
2.1	Gesamtarchitektur	5
2.2	Architekturentscheidungen und Abweichungen vom Projektplan	5
2.3	Komponentenübersicht	6
3	Elektronik	7
3.1	Sensorauswahl	7
3.1.1	GPS-Sensor	7
3.1.2	IMU-Sensor	7
3.1.3	Temperatursensor	8
3.1.4	Drehzahlsensor	8
3.1.5	Mikrocontroller	9
3.1.6	Passivbauteile und Schutzkomponenten	10
3.2	Prototypenaufbau auf dem Breadboard	10
3.3	Planung und Aufbau der Drehzahlmesserschaltung	10
3.4	Verlöten auf Lochrasterplatinen	11
3.5	Spannungsversorgung	11
3.5.1	Anforderungsanalyse	11
3.5.2	Buck-Converter-Auswahl	12
4	Mechanik	12
4.1	Anforderungen und Designziele	12
4.2	CAD-Modellierung	12
4.3	3D Druck	12
4.4	Druckparameter und Materialwahl	13
5	ESP32-Firmware	13
5.1	Softwarestruktur	14
5.2	GPS-Integration	14
5.3	IMU-Integration	14
5.4	Temperaturmessung	14
5.5	Drehzahlmessung	15
5.5.1	Drehzahl-Signalgenerator	16
5.6	Standstill-Erkennung	16
5.7	MQTT-Publishing	16
6	Raspberry Pi – Backend und Infrastruktur	16
6.1	MQTT-Broker (Mosquitto)	17
6.2	Python-Umgebung	17
6.3	Datenbank	17
6.3.1	Datenbankschema	17
6.3.2	RideRecorder	18
6.3.3	Batch-Writes	19
6.4	Autostart per systemd	19
7	Web-Dashboard	20
7.1	Technologiewahl	20
7.2	Architektur: MQTT-Thread und Dash-Callbacks	20

7.3	Live-Ansicht	21
7.4	Fahrtenaufzeichnung und -historie	21
7.5	Entwicklungshilfe: fake_esp.py	22
8	Inbetriebnahme	22
8.1	Regulärer Betrieb	22
8.1.1	Voraussetzungen	22
8.1.2	Systemstart	22
8.1.3	Dashboard aufrufen	22
8.1.4	Fahrtaufzeichnung	23
8.1.5	Systemstatus prüfen	23
8.2	Einrichtung auf einem neuen System	23
8.2.1	Voraussetzungen	23
8.2.2	Raspberry Pi einrichten	23
8.2.3	ESP32 flashen	25
8.2.4	Elektronik und Mechanik	26
9	Aufwandsvergleich und Arbeitsprotokoll	26
9.1	Geplanter vs. tatsächlicher Aufwand	26
9.1.1	Elektronik	27
9.1.2	Mechanik	28
9.1.3	Informatik	28
9.2	Stundenliste	28
10	Commit-Historie	31
11	Probleme und Lösungen	31
11.1	ESP32-Upload-Fehler: TX-Pfad durch GPS-Leitung blockiert	31
11.2	GPS-Phantomgeschwindigkeit bei Stillstand	32
11.3	Dash-Callback-Ausfall bei fehlendem DOM-Element	32
11.4	MQTT-Subscription nach Reconnect verloren	32
11.5	RPM-Signalqualität: Debugging der Schaltung und galvanische Trennung	33
11.5.1	Erste Tests: Rohdaten am GPIO	33
11.5.2	Schmitt-Trigger (74HC14): Mehrfachtrigger beseitigt	33
11.5.3	PC817-Optokoppler: Pegel zu hoch für ESP32-LOW	33
11.5.4	Firmware: ISR-Initialisierung und Plausibilitätsfilter	34
11.6	PT100/PT1000-Verwechslung	34
11.7	Bordnetz-Instabilität beim Motorradstart	35
11.8	Wechsel der Spannungsversorgung: Bordnetz auf Powerbank	35
12	Fazit	36

1 Einleitung

Dieses Dokument beschreibt den Aufbau, die Implementierung und die Inbetriebnahme des MotoSense-Systems – von der Hardwareauswahl und dem Schaltungsdesign bis hin zur Firmware und dem Web-Dashboard.

1.1 Projektbeschreibung und Zielsetzung

MotoSense ist ein nachrüstbares Telemetrie-System für ältere Motorräder, die ab Werk über keine oder nur sehr eingeschränkte elektronische Systeme zur Erfassung von Fahrdaten verfügen. Ziel des Projekts ist es, solche Fahrzeuge möglichst unkompliziert mit Sensorik auszurüsten, ohne dabei in die bestehende Fahrzeugelektronik eingreifen zu müssen. In der aktuellen Version erfasst das System vier Messgrößen: GPS-Position und Geschwindigkeit, Motordrehzahl, Motor-/Öltemperatur sowie den Neigungswinkel des Fahrzeugs. Die Sensordaten werden in Echtzeit erfasst, verarbeitet und über eine Web-App auf dem Smartphone angezeigt, die über das lokale WLAN erreichbar ist.

1.2 Versuchsfahrzeug

Als Versuchsfahrzeug wurde eine Suzuki Bandit 1200 (GV77A), Baujahr 2002, gewählt. Zum einen stand das Motorrad bereits zur Verfügung, zum anderen bietet es ab Werk außer Geschwindigkeit und Drehzahl keine weiteren Fahrdaten – wodurch sich der Mehrwert eines Nachrüstsystems wie MotoSense besonders gut zeigen lässt. Zusätzlich ist die Elektronik des Motorrads sehr simpel aufgebaut, was den Einbau zusätzlicher Sensorik deutlich vereinfacht, da keine Rücksicht auf komplexe Bordelektronik oder Steuergeräte genommen werden muss.

1.3 Abgrenzung zum Projektvorschlag

Der Großteil des ursprünglichen Projektvorschlags wurde wie geplant umgesetzt. Im Laufe des Projekts haben sich jedoch zwei wesentliche Änderungen ergeben: Zum einen wurde die Spannungsversorgung der Mikrocontroller von einer fahrzeugseitigen auf eine externe Versorgung umgestellt, zum anderen wurde das System von einem fest verbauten Aufbau zu einem tragbaren System umkonzipiert. Beide Entscheidungen und die jeweiligen Gründe dafür werden im weiteren Verlauf der Dokumentation genauer erläutert.

2 Systemarchitektur

Dieses Kapitel beschreibt den Aufbau des Gesamtsystems sowie das Zusammenspiel der einzelnen Komponenten von der Sensorebene bis zur Anzeige auf dem Smartphone.

2.1 Gesamtarchitektur

Das Gesamtsystem ist bewusst modular aufgebaut, damit sich in Zukunft ohne großen Aufwand weitere Sensoren oder Geräte einbinden lassen. Die zentrale Rolle übernimmt ein Raspberry Pi 4 Model B. Er hält die persistenten Fahrtdaten in einer SQLite-Datenbank vor und stellt die Live-Werte sowie die Fahrtenhistorie über eine mit Dash umgesetzte Web-App dar. Gleichzeitig läuft auf dem Pi mit Mosquitto der MQTT-Broker, über den sämtliche Kommunikation im System läuft. Um alle Geräte miteinander zu verbinden, spannt der Pi zusätzlich ein eigenes WLAN-Netzwerk auf und fungiert als Access Point für alle weiteren Geräte – sowohl für den ESP32 als auch für das Smartphone, auf dem das Dashboard angezeigt wird. Sämtliche Sensoren sind aktuell an einem einzigen ESP32 angeschlossen, der die Messwerte einliest, aufbereitet und per MQTT an den Raspberry Pi weiterleitet. Da die Kommunikation ausschließlich über MQTT läuft, lassen sich bei Bedarf problemlos weitere Geräte – etwa zusätzliche ESP32 für neue Sensoren – an den Pi anbinden, ohne dass am bestehenden System etwas geändert werden muss. Das System kann auf diese Weise schrittweise um zusätzliche Funktionalität erweitert werden.

2.2 Architekturentscheidungen und Abweichungen vom Projektplan

Im Vergleich zum ursprünglichen Projektvorschlag haben sich im Laufe der Umsetzung einige Abweichungen ergeben.

Stromversorgung: Im Projektvorschlag war vorgesehen, die Mikrocontroller direkt aus dem Bordnetz des Motorrads zu versorgen. In der Praxis hat sich gezeigt, dass die Bordspannung der Bandit 1200 zu stark schwankt, um ESP32 und Raspberry Pi zuverlässig und sicher zu betreiben. Aus diesem Grund wurde auf eine externe Stromversorgung umgestellt.

Mechanischer Aufbau: Ursprünglich war ein fest am Motorrad verbautes System geplant. Stattdessen sitzt das Gesamtsystem nun in einer Tasche am Tank. Das hat zwei Gründe: Zum einen hat der GPS-Sensor dadurch freie Sicht zum Himmel, was für einen zuverlässigen Empfang notwendig ist, zum anderen lässt sich das System dadurch auch unkompliziert an anderen Motorrädern einsetzen, statt fest an ein einziges Fahrzeug gebunden zu sein.

Anzahl der Mikrocontroller: Im Projektvorschlag war noch offen, ob ein oder mehrere ESP32 zum Einsatz kommen. In der finalen Umsetzung wurden alle Sensoren an einem einzigen ESP32 angeschlossen, was den Aufbau vereinfacht und für die aktuelle Sensoranzahl ausreichend Rechenleistung und Pins bietet.

Temperatursensorik: Ursprünglich war ein PT1000-Sensor vorgesehen. Bei der Inbetriebnahme des MAX31865-Boards stellte sich heraus, dass die verwendete Platine für einen PT100-Sensor ausgelegt ist (Referenzwiderstand $430\ \Omega$ statt $4300\ \Omega$). Entsprechend wurde auf einen PT100-Sensor umgestellt und die Firmware-Parameter angepasst.

2.3 Komponentenübersicht

Komponente	Schnittstelle	Funktion / Rolle
ESP32 DevKit (WROOM-32)	–	Zentraler Sensor-Knoten; liest alle Sensoren aus, bündelt die Werte zu einem JSON-Paket und sendet sie per MQTT an den Raspberry Pi
GY-NEO6MV2 (NEO-6M)	UART (UART2, GPIO16/17)	GPS-Modul zur Erfassung von Position, Geschwindigkeit und Satellitenanzahl
MPU-6050 (GY-521)	I2C (GPIO21/22)	IMU zur Erfassung von Schräglage und Neigung; liefert Roll- und Pitchwinkel über einen Komplementärfilter
MAX31865	SPI (VSPI)	RTD-zu-Digital-Wandler zur Auswertung des PT100-Temperatursensors
PT100-Sensor	2-Leiter, an MAX31865	Messung der Öltemperatur; Referenzwiderstand $R_{ref} = 430 \, \Omega$, $R_{nominal} = 100 \, \Omega$
74HC14 (Schmitt-Trigger)	Digitaleingang	Formt den verrauschten induzierten Drehzahlimpuls zu einer sauberen Rechteckflanke
PC817 (Optokoppler)	Digitaleingang, GPIO27	Galvanische Trennung zwischen Zündanlage und ESP32-Logik
Induktiver Pickup	Analog, am Zündkabel	Berührungslose Abnahme des Zündimpulses zur Drehzahlbestimmung
Raspberry Pi 4 Model B	WLAN, Ethernet	Zentrale Recheneinheit; MQTT-Broker, Access Point, Datenhaltung und Bereitstellung des Dashboards
Smartphone	WLAN	Anzeige des Live-Dashboards und der Fahrtenhistorie im Browser

Tabelle 1: Komponentenübersicht

3 Elektronik

Dieses Kapitel beschreibt die Auswahl der elektronischen Komponenten sowie den Aufbau und die Inbetriebnahme der einzelnen Schaltungen, vom ersten Test auf dem Breadboard bis zur fertig verlöteten Platine.

3.1 Sensorauswahl

Die Auswahl der einzelnen Komponenten orientierte sich vor allem an drei Kriterien: Verfügbarkeit, Kosten und dem Implementierungsaufwand. Da das Projekt im Rahmen eines begrenzten Budgets und Zeitrahmens umgesetzt wurde, fiel die Wahl bei den meisten Bauteilen auf gut dokumentierte und weit verbreitete Module, für die ausreichend Bibliotheken und Communityressourcen verfügbar sind.

3.1.1 GPS-Sensor

Als GPS-Sensor kommt das Modul GY-NEO6MV2 zum Einsatz, das auf dem u-blox NEO-6M GPS-Chip basiert. Das Modul benötigt eine Betriebsspannung von 3,3 V und kommuniziert über eine UART-Schnittstelle mit dem ESP32. Die Positionsgenauigkeit liegt bei rund 2,5 m, was für die Zwecke des Projekts völlig ausreichend ist. Der größte Nachteil des Moduls zeigte sich in der Praxis: Bis nach dem Einschalten verwertbare Positionsdaten geliefert werden, vergeht eine vergleichsweise lange Zeit.

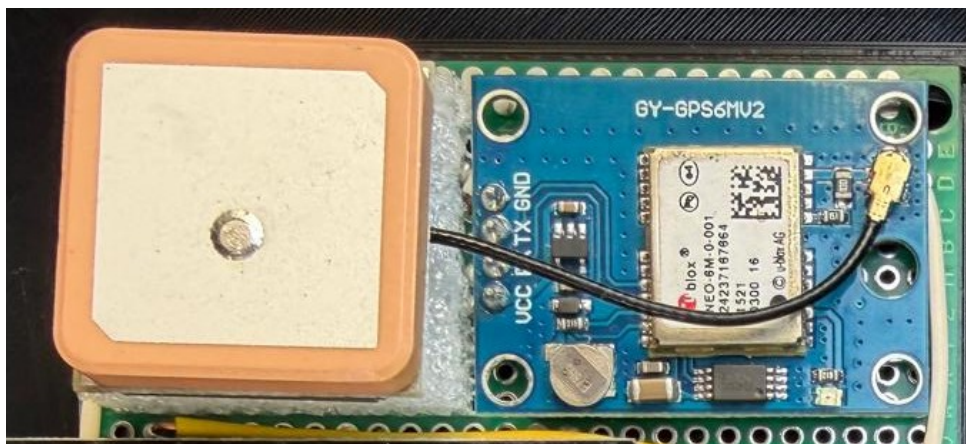


Abbildung 1: GPS Sensor

3.1.2 IMU-Sensor

Zur Messung der Neigung wird ein MPU-6050 eingesetzt, ein 6-Achsen-IMU (Inertial Measurement Unit), das einen 3-Achsen-Beschleunigungssensor und ein 3-Achsen-Gyroskop in einem Chip vereint. Der Sensor arbeitet mit einer Betriebsspannung von 3,3 V und kommuniziert über die I2C-Schnittstelle mit dem Mikrocontroller.

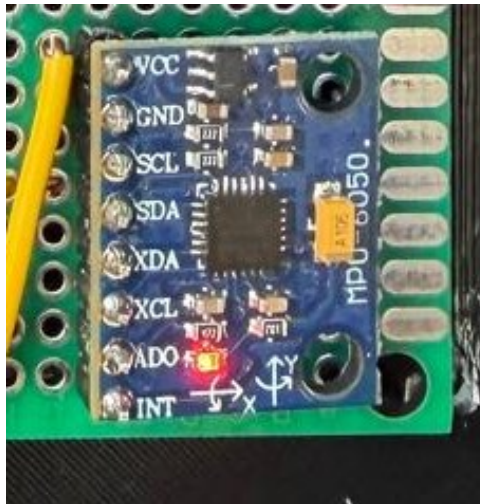


Abbildung 2: IMU Sensor

3.1.3 Temperatursensor

Zur Messung der Öltemperatur kommt ein MAX31865 in Kombination mit einem PT100-Sensor zum Einsatz. Der MAX31865 ist ein RTD-zu-Digital-Wandler, der den Widerstand des PT100 ausliest und intern in eine Temperatur umrechnet. Der Baustein arbeitet mit einer Betriebsspannung von 3,3 V und kommuniziert über die SPI-Schnittstelle mit dem Mikrocontroller.

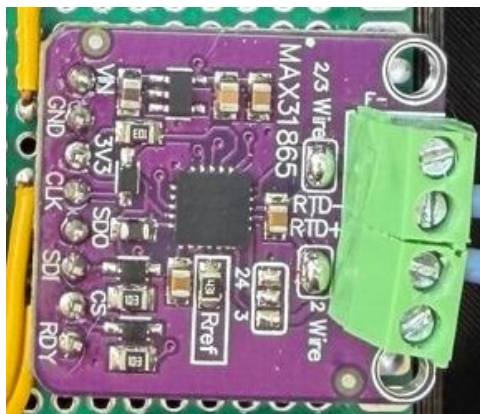


Abbildung 3: Temperatur Sensor

Der PT100 ist ein Platin-Widerstandsthermometer mit einem Widerstand von $100\ \Omega$ bei $0\ ^\circ\text{C}$, der mit steigender Temperatur näherungsweise linear zunimmt. PT100-Sensoren zählen zu den genauesten und stabilsten RTD-Typen und sind damit deutlich präziser als beispielsweise ein NTC-Thermistor. Angeschlossen wird der Sensor in 2-Leiter-Technik, was aufgrund der kurzen Leitungslänge im Projekt ausreichend genau ist.

3.1.4 Drehzahlsensor

Zur Erfassung der Motordrehzahl wird ein induktiver Pickup eingesetzt, der berührungslos am Zündkabel angebracht wird. Da sich kein passender fertiger Sensor für diesen Anwendungsfall finden ließ, wurde die Lösung selbst entwickelt und aufgebaut: Dafür werden ca. 15 Windungen Draht um das Zündkabel gewickelt, wodurch der Zündimpuls induktiv abgegriffen werden kann, ohne in die Zündanlage selbst eingreifen zu müssen.

Die Bandit 1200 verfügt über eine Wasted-Spark-Zündung mit zwei Doppelzündspulen, bei der jede Kerze einmal pro Kurbelwellenumdrehung zündet. Bei Abnahme an einem einzelnen Zündkabel

ergibt sich dadurch genau ein Puls pro Kurbelwellenumdrehung, woraus sich die Drehzahl direkt berechnen lässt: $\text{RPM} = \text{Pulse pro Sekunde} \times 60$.

Da das am Zündkabel abgegriffene Rohsignal stark verrauscht und mit hohen Spannungsspitzen behaftet ist, wird es vor der Weiterverarbeitung aufbereitet. Ein Schmitt-Trigger (74HC14) formt das Signal zu einer sauberen Rechteckflanke, ein nachgeschalteter Optokoppler (PC817) trennt den ESP32 anschließend galvanisch vollständig von der Zündanlage. Erst danach gelangt das Signal auf Logikebene (3,3 V) an einen Interrupt-Eingang des ESP32, über den die Pulse ausgewertet werden.

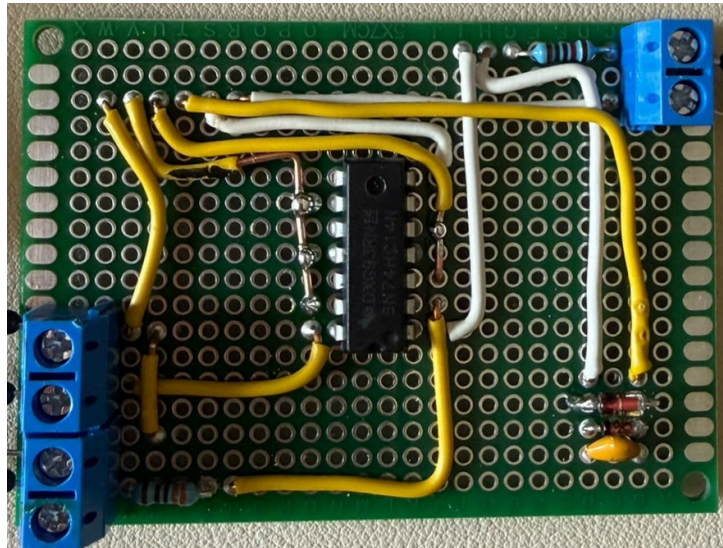


Abbildung 4: RPM Sensor

3.1.5 Mikrocontroller

Für die Kombination aus ESP32 als MQTT-Client und Raspberry Pi als MQTT-Broker konnte auf Erfahrung aus früheren Projekten zurückgegriffen werden, in denen sich dieser Aufbau bereits bewährt hatte. Zusätzlich waren beide Mikrocontroller bereits vorhanden, wodurch sich die Anschaffungskosten für das Projekt reduzierten.

Die Entscheidung für MQTT als Kommunikationsprotokoll ergibt sich direkt aus der angestrebten Modularität des Systems: Da Sender und Empfänger bei MQTT räumlich und zeitlich voneinander entkoppelt sind, können ESP32 und Raspberry Pi unabhängig voneinander arbeiten, solange beide mit dem Broker verbunden sind. Gleichzeitig lassen sich dadurch jederzeit weitere Sensoren oder zusätzliche Mikrocontroller in das System einbinden, ohne dass an der bestehenden Kommunikation etwas geändert werden muss.

Der ESP32 (WROOM-32) arbeitet mit einer Betriebsspannung von 3,3 V, kann aber über den Onboard-Spannungsregler auch direkt mit 5 V versorgt werden. Er verfügt über einen Dual-Core-Prozessor mit bis zu 240 MHz Taktfrequenz, was für die parallele Erfassung mehrerer Sensoren über unterschiedliche Schnittstellen mehr als ausreichend ist. Ein großer Vorteil des ESP32 ist die hohe Anzahl an verfügbaren GPIOs sowie mehrere unabhängige UART-, I2C- und SPI-Schnittstellen, wodurch GPS, IMU und Temperatursensor gleichzeitig betrieben werden können, ohne sich gegenseitig zu blockieren. Zusätzlich bringt der ESP32 WLAN bereits integriert mit, was ihn für die drahtlose MQTT-Anbindung an den Raspberry Pi prädestiniert.

Der Raspberry Pi 4 Model B wird über USB-C versorgt und arbeitet intern mit einer Betriebsspannung von 5 V. Mit seinem Quad-Core-Prozessor und entsprechend mehr Rechenleistung als ein Mikrocontroller wie der ESP32 ist er in der Lage, mehrere Dienste gleichzeitig zu betreiben: den MQTT-Broker, die Datenhaltung in der SQLite-Datenbank, das Backend sowie den Webserver

für das Dashboard. Da auf dem Pi ein vollwertiges Linux läuft, lassen sich diese Komponenten unkompliziert in Python umsetzen. Über die integrierte WLAN-Schnittstelle kann der Pi außerdem selbst als Access Point für das gesamte System fungieren.

3.1.6 Passivbauteile und Schutzkomponenten

Neben den Hauptkomponenten kommen in der Drehzahlschaltung mehrere Passivbauteile zum Einsatz, die in erster Linie dem Schutz der nachfolgenden Stufen sowie der Signalaufbereitung dienen. Am Eingang begrenzt ein Widerstand (R_1 , 10 k Ω) den Strom, während eine Zenerdiode (D_1 , 3,6 V) das Signal nach oben hin klemmt und so Überspannungen abfängt. Für die negative Klemmung kommt eine schnelle Schaltdiode (D_2 , 1N4148) zum Einsatz – bewusst keine gewöhnliche Gleichrichterdiode aus der 1N400x-Reihe, da diese für die auftretenden Schaltfrequenzen zu langsam wäre. Ein kleiner Keramik Kondensator (C_1 , 1–10 nF) sorgt zusätzlich für eine Entstörung des Eingangssignals.

Am Schmitt-Trigger selbst stabilisiert ein Abblockkondensator (C_2 , 100 nF) die Versorgungsspannung. Auf der Optokoppler-Seite begrenzt ein Vorwiderstand (R_2 , 330 Ω) den Strom durch die interne LED, während ein Pull-up-Widerstand (R_3 , 10 k Ω) am Ausgang den High-Pegel für den ESP32-Eingang definiert.

In den Signalpfaden wurden durchgehend Keramik Kondensatoren statt Elektrolytkondensatoren verwendet, da diese stabiler und unempfindlicher gegenüber Temperaturschwankungen sind. Bei der Auswahl aller Passivbauteile wurde grundsätzlich auf gängige, gut verfügbare Standardwerte geachtet, um Bestellaufwand und Kosten möglichst gering zu halten.

3.2 Prototypenaufbau auf dem Breadboard

Der Aufbau der Sensorik erfolgte schrittweise und Sensor für Sensor auf dem Breadboard, in der Reihenfolge IMU, GPS, Temperatursensor und zuletzt Drehzahlmessung. Ein Sensor galt dabei erst als fertig getestet, sobald über den Serial Monitor des ESP32 plausible und korrekte Messwerte ausgelesen werden konnten.

Erst nachdem alle Sensoren auf dem ESP32 zuverlässig funktionierten, wurde dieser per Kabel direkt mit dem Raspberry Pi verbunden und die zugehörige Software auf dem Pi entwickelt. Im Anschluss daran wurde die Datenbank zur Speicherung der Fahrtdaten ergänzt, bevor die Verbindung zwischen ESP32 und Raspberry Pi schließlich auf MQTT umgestellt wurde.

3.3 Planung und Aufbau der Drehzahlmesserschaltung

Da sich am Markt kein passender fertiger Sensor für die induktive Drehzahlabnahme an einem Zündkabel finden ließ, musste die Schaltung von Grund auf selbst entwickelt werden. Die grundlegende Topologie – Pickup, Strombegrenzung und Klemmung, Schmitt-Trigger und Optokoppler – wurde dabei durch Recherche im Internet erarbeitet, da diese Kombination als robuste und in der Praxis bewährte Lösung für die Aufbereitung verrauschter induktiver Signale gilt.

Besonderer Wert wurde bei der Planung auf die Sicherheit der Schaltung gelegt. Da am Zündkabel hohe und schwer vorhersagbare Spannungsspitzen auftreten können, war eine galvanische Trennung zwischen Zündanlage und ESP32 unverzichtbar. Der Optokoppler übernimmt diese Aufgabe, indem er die Zündungsseite komplett elektrisch vom Mikrocontroller entkoppelt und so eine Beschädigung des ESP32 zuverlässig ausschließt. Vorgeschaltet sorgen Strombegrenzungswiderstand sowie Zener- und Schaltdiode dafür, dass bereits vor dem Schmitt-Trigger keine schädlichen Spannungspegel anliegen können.

Um die Schaltung gefahrlos und unabhängig vom Motorrad entwickeln und testen zu können, wurde während der gesamten Entwicklungsphase auf reale Zündsignale verzichtet. Stattdessen kam

ein zweiter ESP32 als Funktionsgenerator zum Einsatz, der über direktes Toggeln eines GPIO-Pins Rechtecksignale mit definierbarer Frequenz erzeugte. Dadurch ließ sich das Verhalten der Schaltung – von der Signalformung am Schmitt-Trigger bis zur sauberen Pulsausgabe nach dem Optokoppler – bei verschiedenen, reproduzierbaren Drehzahlen am Oszilloskop bzw. über den Serial Monitor überprüfen, bevor die Schaltung später am tatsächlichen Zündsignal des Motorrads zum Einsatz kam.

3.4 Verlöten auf Lochrasterplatinen

Für den finalen Aufbau standen mehrere vordefinierte Lochrasterplatinengrößen zur Auswahl, wobei jeweils die kleinstmögliche Größe gewählt wurde, die noch alle benötigten Bauteile aufnehmen konnte. Insgesamt wurden zwei separate Platinen verlötet: eine für den ESP32 mit den angeschlossenen Sensoren und eine zweite für die Drehzahlmesserschaltung.

Durch die Anzahl der verbauten Sensoren auf der Hauptplatine war der verfügbare Platz entsprechend knapp bemessen, sodass beim Layout auf eine möglichst platzsparende Anordnung der Bauteile geachtet werden musste. Größere Probleme beim Lötprozess selbst gab es darüber hinaus nicht.

Nach dem Verlöten wurden sämtliche Signalwege mit einem Multimeter durchgemessen, um mögliche Lötfehler wie Kurzschlüsse oder unterbrochene Verbindungen frühzeitig auszuschließen, bevor die Platinen in Betrieb genommen wurden.

3.5 Spannungsversorgung

3.5.1 Anforderungsanalyse

Vor der Auslegung der Spannungsversorgung wurden zunächst die Anforderungen der einzelnen Komponenten ermittelt. Der ESP32 benötigt 3,3 V und versorgt darüber gleichzeitig Temperatursensor, IMU und GPS-Modul. Der Raspberry Pi benötigt 5 V. Die Drehzahlmesserschaltung wird ebenfalls mit 3,3 V betrieben, jedoch bewusst getrennt von der Versorgung der Mikrocontroller, um die galvanische Trennung der Schaltung nicht zu untergraben.

Um am Motorrad ausreichend Anschlussmöglichkeiten für die Spannungsversorgung der einzelnen Komponenten zu haben, wurde zusätzlich ein eigener Stromverteiler gebaut: Das Gehäuse dafür wurde 3D-gedruckt und selbst verkabelt, mit insgesamt vier möglichen Anschlüssen für Spannung und Masse. Welche Komponenten letztlich tatsächlich über das Bordnetz versorgt werden, wird im Kapitel „Probleme und Lösungen“ noch genauer erläutert.

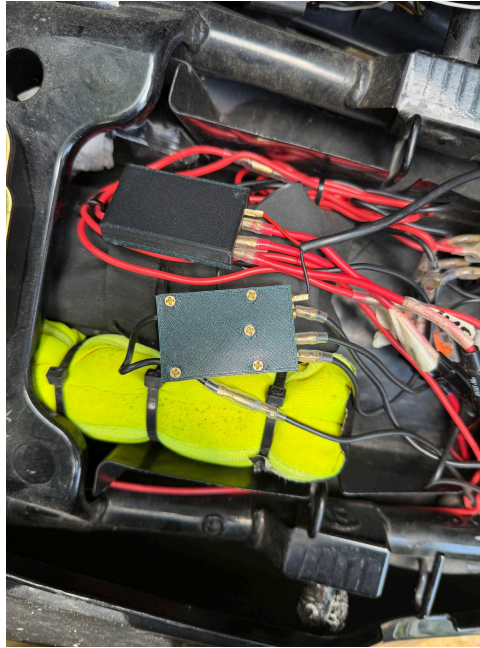


Abbildung 5: Stromverteiler unter Sitz

3.5.2 Buck-Converter-Auswahl

Für die Spannungsversorgung der 3,3-V-Ebene kommt ein LM2596-Buck-Converter zum Einsatz, der die Eingangsspannung auf den benötigten Pegel herunterregelt. Da ESP32 und Raspberry Pi mittlerweile über eine externe Spannungsquelle versorgt werden, wird der Buck-Converter aktuell ausschließlich für die Versorgung der Drehzahlmesserschaltung am Bordnetz genutzt.

4 Mechanik

Dieses Kapitel beschreibt die mechanische Umsetzung von MotoSense, von den Anforderungen an die Gehäuse über die CAD-Modellierung bis zur Fertigung und Montage am Fahrzeug.

4.1 Anforderungen und Designziele

Bei der mechanischen Planung stand vor allem der Schutz der Sensoren und Mikrocontroller vor äußeren Einflüssen im Vordergrund. Eine Witterungsfestigkeit der Gehäuse war dabei nicht erforderlich, da die Hauptkomponenten im Tankrucksack untergebracht sind und die Drehzahlmesserschaltung unter dem Sitz verbaut wird – beide Bereiche sind somit ohnehin vor direkter Witterung geschützt. Ein weiteres Designziel war eine möglichst einfache Zugänglichkeit, etwa um Anschlüsse zu prüfen oder Komponenten bei Bedarf auszutauschen.

4.2 CAD-Modellierung

Für die Modellierung der Gehäuse wurde Fusion 360 verwendet, da über eine Studienlizenz bereits Zugriff darauf bestand. Als Grundlage für die Modellierung wurden die jeweiligen Platinen ausgemessen und die Position der herausführenden Kabel festgelegt, bevor die eigentlichen Gehäuse konstruiert wurden. Bis auf kleinere nachträgliche Anpassungen verlief die Modellierung dabei weitgehend problemlos.

4.3 3D Druck

Insgesamt wurden drei Gehäuse konstruiert und gedruckt: eines für die Hauptplatine mit ESP32 und Sensoren, eines für die Drehzahlmesserschaltung und eines für den Stromverteiler. Nach dem Druck wurden die jeweiligen Platinen in die Gehäuse eingeschraubt. Der Stromverteiler wird über ein

Relais angesteuert, das auf der Zündung sitzt und unter dem Sitz verbaut ist. Dadurch schaltet die Versorgung der Drehzahlmesserschaltung automatisch mit der Zündung des Motorrads.



Abbildung 6: Strom Verteiler



Abbildung 7: Case des Sensorboards

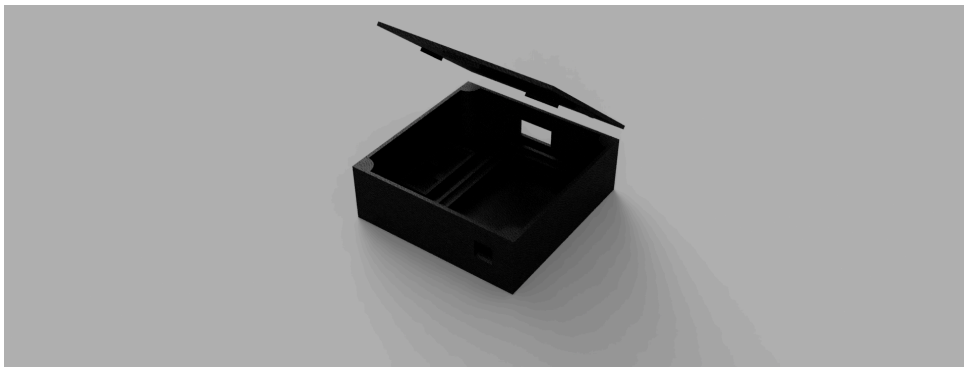


Abbildung 8: Case des RPM Boards

4.4 Druckparameter und Materialwahl

Für alle drei Gehäuse wurde PLA als Druckmaterial verwendet. Die Drucktemperatur lag bei 230 °C, die Druckbetttemperatur bei 70 °C.

5 ESP32-Firmware

Die Firmware läuft auf dem ESP32-WROOM-32 unter dem Arduino-Framework, verwaltet mit PlatformIO. Sie liest alle vier Sensoren aus, filtert die Rohdaten und publisht alle 100 ms einen JSON-Datensatz über MQTT an den Raspberry Pi.

5.1 Softwarestruktur

Der gesamte Firmware-Code ist in einer einzigen Datei `main.cpp` organisiert. Diese Struktur ist für ein eingebettetes System dieser Größe üblich – eine Aufteilung in mehrere Module würde keinen nennenswerten Vorteil bringen.

`setup()` initialisiert alle Peripherien in fester Reihenfolge: erst UART für GPS, dann I²C für die IMU mit anschließender Kalibrierung, dann SPI für den Temperatursensor, zuletzt den Interrupt-Pin für die Drehzahlmessung und die WLAN-Verbindung. `loop()` ist bewusst nicht-blockierend aufgebaut – kein `delay()`, keine wartenden Schleifen. Stattdessen wird jede wiederkehrende Aufgabe über einen `millis()`-Zeitvergleich gesteuert:

Aufgabe	Intervall	Begründung
GPS-UART leeren	jeden <code>loop()</code> -Durchlauf	Puffer darf nicht überlaufen
IMU <code>update()</code>	jeden <code>loop()</code> -Durchlauf	Komplementärfilter braucht dichte Updates
Temperatur lesen	1 Hz	<code>temperature()</code> blockiert 70 ms (1-Shot-Wandlung)
JSON publishen	100 ms (10 Hz)	Senderate Dashboard
MQTT reconnect	alle 2 s bei Verbindungsabbruch	nicht-blockierendes Retry

Tabelle 2: Zeitsteuerung der `loop()`-Aufgaben

5.2 GPS-Integration

Das NEO-6M-Modul ist an UART2 des ESP32 angebunden (RX = GPIO 16, TX = GPIO 17) und wird mit 9600 Baud betrieben. Die NMEA-Sätze verarbeitet die Bibliothek `TinyGPSPlus`.

In jedem `loop()`-Durchlauf wird der UART-Puffer vollständig geleert:

```
while (gpsSerial.available() > 0) gps.encode(gpsSerial.read());
```

Dieses nicht-blockierende Muster ist entscheidend: Ein `readStringUntil('\n')` würde den ESP32 für die Dauer eines kompletten NMEA-Satzes blockieren und dabei sowohl den IMU-Komplementärfilter als auch den RPM-Interrupt-Handler ausbremsen. GPS-Koordinaten und Geschwindigkeit werden nur dann übernommen, wenn `isValid()` `true` zurückgibt – bis dahin bleiben die zuletzt bekannten Werte erhalten, sodass das Dashboard immer einen vollständigen JSON-Datensatz empfängt.

5.3 IMU-Integration

Der MPU-6050 läuft über I²C (SDA = GPIO 21, SCL = GPIO 22). Verwendet wird die Bibliothek `MPU6050_light`, die einen integrierten Komplementärfilter mitbringt. Dieser kombiniert das Gyroskopsignal (präzise auf kurzen Zeitskalen, aber driftbehaftet) mit dem Beschleunigungsvektor (langzeitstabil, aber rauschempfindlich) und liefert damit stabile Winkel auch bei Beschleunigung und Bremsen.

`mpu.calcOffsets()` wird einmalig in `setup()` aufgerufen, während das Board ruhig und flach liegt. Das Motorrad muss für die endgültige Kalibrierung in seiner normalen Standposition stehen. `mpu.update()` muss in jedem `loop()`-Durchlauf aufgerufen werden, da der Filter ohne regelmäßige Aktualisierungen driftet.

5.4 Temperaturmessung

Der MAX31865-Wandler kommuniziert über Software-SPI (CS = GPIO 5, SCK = GPIO 18, MISO = GPIO 19, MOSI = GPIO 23). Der Sensor wird nur einmal pro Sekunde ausgelesen, da `temperature()`

eine 1-Shot-Wandlung auslöst und dabei rund 70 ms blockiert – bei 10-Hz-Aufruf würde das die gesamte Timing-Struktur der Firmware zerstören.

Nach jeder Messung wird ein zweistufiger Plausibilitätscheck durchgeführt. Zunächst wird der Fault-Status des MAX31865 abgefragt, der Leitungsbrüche und Kurzschlüsse an der Sonde erkennt. Danach wird geprüft, ob der korrigierte Temperaturwert innerhalb des zulässigen Bereichs von -50 °C bis 250 °C liegt. Nur fehlerfreie und plausible Messwerte werden in `curTemp` übernommen – andernfalls bleibt der zuletzt gültige Wert stehen.

```
float   tRaw      = rtd.temperature(RNOMINAL, RREF);
float   tKorr     = tRaw + TEMP_OFFSET;
uint8_t fault     = rtd.readFault();
bool    implausibel = (tKorr < TEMP_MIN || tKorr > TEMP_MAX);
if (fault) rtd.clearFault();
if (!fault && !implausibel) curTemp = tKorr;
```

Ein fester Kalibrierungsoffset (`TEMP_OFFSET = -10.0`) wird zur Rohtemperatur addiert, um den Einbauort der Sonde zu kompensieren.

5.5 Drehzahlmessung

Der Optokoppler-Ausgang liegt an GPIO 27 und löst bei jeder fallenden Flanke einen Hardware-Interrupt aus. Die ISR `onRpmPulse()` ist mit `IRAM_ATTR` markiert und liegt damit im internen RAM des ESP32 – notwendig, damit der Interrupt auch während eines Flash-Zugriffs (z. B. beim MQTT-Senden) zuverlässig ausgeführt wird.

Die ISR implementiert zwei aufeinanderfolgende Filter:

Entprellung: Intervalle unter $2000\text{ }\mu\text{s}$ werden verworfen. Das entspricht einer physikalisch unmöglichen Drehzahl von über 30.000 1/min und schließt hochfrequente Störimpulse aus.

Plausibilitätsfilter: Ein neues Intervall wird nur akzeptiert, wenn es mindestens 60 % des vorherigen gültigen Intervalls beträgt. Eine Unterschreitung würde eine Drehzahlverdopplung innerhalb einer einzigen Kurbelwellenumdrehung bedeuten, was mechanisch nicht möglich ist. Dieser Filter unterdrückt einzelne Störimpulse zuverlässig, ohne echte Beschleunigungsvorgänge zu beeinträchtigen.

```
if (lastGoodIntervalUs > 0 && dt * 10UL < lastGoodIntervalUs * 6UL) {
    return;
}
```

`readRpm()` liest die volatile Variablen atomar unter `noInterrupts()` und gibt 0 zurück, wenn seit dem letzten Puls mehr als 500 ms vergangen sind (Stall-Timeout) – bei stehendem Motor gibt es keine Interrupts, und ohne Timeout würde die letzte gemessene Drehzahl dauerhaft angezeigt bleiben.

Eine kritische Initialisierungsdetail: `lastPulseUs` wird in `setup()` mit `micros()` vorbelegt, nicht mit 0. Würde die Variable den Wert 0 behalten, hätte der allererste Puls nach dem Systemstart ein Intervall von mehreren Millionen Mikrosekunden – das entspräche rechnerisch einer Drehzahl nahe Null und würde als neuer `lastGoodIntervalUs`-Referenzwert gespeichert. Alle nachfolgenden echten Pulse würden dann als zu kurz und damit als implausibel verworfen, bis das Referenzintervall endlich überschrieben wird. Die Folge wäre, dass die Drehzahlanzeige nach dem Start für mehrere Sekunden auf Null eingefroren bleibt.

5.5.1 Drehzahl-Signalgenerator

Zur Entwicklung und zum Test der Interrupt-Logik ohne laufenden Motor wurde ein eigenständiger Signalgenerator auf einem zweiten ESP32 aufgebaut (`function_generator.cpp`).

Der Generator erzeugt auf GPIO 25 ein Rechtecksignal, dessen Frequenz einer vorgegebenen Drehzahl entspricht. Die Signalerzeugung erfolgt über Software-Toggle mit `micros()` – bewusst nicht über das LEDC-PWM-Modul des ESP32, da LEDC bei sehr niedrigen Frequenzen (Leerlauf: 0,5 Hz) unzuverlässig rundet und teils klemmt. Die halbe Periode berechnet sich zu:

$$T_{\text{half}} = \frac{30 \cdot 10^6 \text{ } \mu\text{s}}{n}$$

wobei n die gewünschte Drehzahl in 1/min ist. Der Ausgang wird über einen 1-kΩ-Vorwiderstand an den Eingang der Signalkette (Knoten K1, vor dem 74HC14) angeschlossen.

Der Generator bietet zwei Betriebsmodi, umschaltbar über den Serial Monitor:

Auto-Sweep: Der Standardmodus nach dem Start. Die Drehzahl durchläuft automatisch den Bereich von 0 bis 11.000 1/min in 500er-Schritten, jeweils 2 s pro Stufe, und fährt anschließend wieder herunter. Damit lässt sich die Reaktion der Interrupt-Auswertung auf den gesamten Drehzahlbereich systematisch überprüfen.

Manueller Modus: Eine Zahl gefolgt von Enter hält die Drehzahl fest auf dem eingegebenen Wert. Mit a + Enter kehrt der Generator zum Auto-Sweep zurück. Dieser Modus eignet sich besonders, um das Verhalten bei Leerlaufdrehzahl (1050 1/min) oder bei Grenzdrehzahlen gezielt zu testen.

5.6 Standstill-Erkennung

Bei stehendem Fahrzeug kann das GPS-Modul durch Messrauschen eine Scheingeschwindigkeit von 1–3 km/h liefern, obwohl das Motorrad steht. Um diese Phantomwerte zu unterdrücken, wird die GPS-Geschwindigkeit nur dann übernommen, wenn das Fahrzeug tatsächlich in Bewegung ist.

Als Kriterium dient der Betrag des Beschleunigungsvektors der IMU: Übersteigt er einen Schwellwert, ist das Fahrzeug in Bewegung, und GPS-Geschwindigkeitswerte werden akzeptiert. Im Stillstand wird `curSpeed` auf 0,0 gesetzt. Die IMU reagiert dabei deutlich schneller und zuverlässiger als das GPS selbst auf Bewegungsbeginn und -ende.

5.7 MQTT-Publishing

Alle 100 ms wird ein JSON-Objekt zusammengebaut und auf dem Topic `motosense/data` veröffentlicht:

```
snprintf(payload, sizeof(payload),
    "{ \"ts\":%lu, \"rpm\":%d, \"speed\":%.1f, \"lat\":%.6f, \"lng\":%.6f, \"sats\":%d, \"roll\":%.1f, \"pitch\":%.1f, \"temp\":%.1f }",
    millis(), rpm, curSpeed, curLat, curLng, curSats, roll, pitch, curTemp);
```

`snprintf` in einen lokalen char-Puffer ist auf eingebetteten Systemen der übliche Weg für JSON ohne externe Bibliothek – String-Konkatenation würde unnötig Heap-Fragmentierung verursachen.

Die MQTT-Verbindung wird über `PubSubClient` verwaltet. Fällt die Verbindung weg, versucht die Firmware alle 2 s einen Reconnect, ohne dabei die GPS- und IMU-Verarbeitung zu blockieren. Das WLAN wird bei Verlust alle 5 s mit `WiFi.reconnect()` neu aufgebaut.

6 Raspberry Pi – Backend und Infrastruktur

Der Raspberry Pi 4 übernimmt drei Aufgaben gleichzeitig: Er betreibt den MQTT-Broker, der die Nachrichten des ESP32 entgegennimmt, er speichert alle Fahrtdaten in einer SQLite-Datenbank, und

er stellt das Web-Dashboard bereit. Alle drei Dienste starten automatisch beim Booten und laufen ohne Bildschirm oder Tastatur.

6.1 MQTT-Broker (Mosquitto)

Als Broker wird Mosquitto eingesetzt. Die Installation erfolgt über den Paketmanager:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Standardmäßig hört Mosquitto nur auf localhost und lehnt externe Verbindungen ab – der ESP32 käme damit nicht ran. Eine eigene Konfigurationsdatei öffnet den Broker für das lokale Netz:

```
/etc/mosquitto/conf.d/motosense.conf
listener 1883
allow_anonymous true
```

`allow_anonymous true` bedeutet, dass keine Authentifizierung erforderlich ist. Für ein abgeschlossenes Insel-Netz am Motorrad, das keinen Internetzugang hat, ist das ausreichend. Nach dem Anlegen der Datei muss der Broker neu gestartet werden:

```
sudo systemctl restart mosquitto
```

Der Funktionstest erfolgt mit den mitgelieferten Kommandozeilenwerkzeugen: In einem Terminal abonniert `mosquitto_sub -h localhost -t "motosense/test"` das Testtopic, im zweiten Terminal sendet `mosquitto_pub -h localhost -t "motosense/test" -m "hallo"` eine Nachricht. Erscheint die Nachricht im ersten Terminal, ist der Broker betriebsbereit.

6.2 Python-Umgebung

Auf aktuellem Raspberry Pi OS (Bookworm) ist das System-Python durch PEP 668 geschützt – `pip install` ohne weiteres schlägt fehl. Die Arbeit erfolgt daher in einem virtuellen Environment:

```
mkdir ~/motosense && cd ~/motosense
python3 -m venv venv
source venv/bin/activate
pip install paho-mqtt dash dash-leaflet plotly
```

Nach `source venv/bin/activate` erscheint `(venv)` vor dem Prompt. Das `venv` muss beim manuellen Start eines Skripts immer aktiv sein; für den automatischen Start per `systemd` wird direkt auf den `venv`-Interpreter gezeigt, sodass kein vorheriges `activate` nötig ist.

6.3 Datenbank

Die Fahrtdaten werden in einer SQLite-Datenbank (`motosense.db`) gespeichert, die im Projektverzeichnis liegt. SQLite benötigt keinen separaten Datenbankdienst und ist damit für den eingebetteten Betrieb auf dem Raspberry Pi gut geeignet. Das Schema und die gesamte Datenbanklogik sind in `db.py` gekapselt.

6.3.1 Datenbankschema

Die Datenbank besteht aus zwei Tabellen:

Spalte	Typ	Bedeutung
id	INTEGER	Primärschlüssel, automatisch vergeben
name	TEXT	Automatisch generierter Name (z. B. „Fahrt 21.06. 14:32“)
started_at	REAL	Unix-Zeitstempel Fahrtbeginn
ended_at	REAL	Unix-Zeitstempel Fahrtende, NULL bei laufender Fahrt
duration_s	REAL	Reine Fahrtzeit in Sekunden, Pausen abgezogen
distance_km	REAL	Zurückgelegte Strecke in km (Haversine-Summe)
max_speed	REAL	Höchstgeschwindigkeit in km/h
max_rpm	INTEGER	Maximale Drehzahl
max_lean_l	REAL	Maximale Schräglage links in Grad
max_lean_r	REAL	Maximale Schräglage rechts in Grad
point_count	INTEGER	Anzahl gespeicherter Trackpoints

Tabelle 3: rides – eine Zeile pro Fahrt

Spalte	Typ	Bedeutung
id	INTEGER	Primärschlüssel
ride_id	INTEGER	Fremdschlüssel auf rides.id
t	REAL	Sekunden seit Fahrtbeginn
lat	REAL	GPS-Breitengrad
lng	REAL	GPS-Längengrad
speed	REAL	Geschwindigkeit in km/h
rpm	INTEGER	Drehzahl
roll	REAL	Schräglage in Grad
pitch	REAL	Nickwinkel in Grad
temp	REAL	Öltemperatur in °C

Tabelle 4: trackpoints – ein Messpunkt je 200ms

Auf `trackpoints.ride_id` liegt ein Index (`idx_tp_ride`), damit die Abfrage aller Punkte einer Fahrt auch bei großen Datenmengen schnell bleibt.

6.3.2 RideRecorder

Der `RideRecorder` ist eine Klasse, die den gesamten Lebenszyklus einer Fahraufnahme verwaltet. Eine globale Instanz `recorder` wird beim Import von `db.py` angelegt und von `dashboard.py` verwendet.

Der Recorder kennt drei Zustände: `idle` (keine aktive Fahrt), `recording` (Punkte werden gespeichert) und `paused` (Fahrt unterbrochen, eingehende Punkte werden verworfen). Der typische Ablauf ist:

`start()` legt eine neue Zeile in `rides` an und wechselt in den Zustand `recording`. `pause()` sichert die bis dahin verstrichene Fahrtzeit in `_accumulated` und schreibt den Pufferinhalt in die Datenbank. `resume()` setzt die Aufnahme fort, indem `_active_since` auf den aktuellen Zeitstempel gesetzt wird. `stop()` addiert die letzte Zeitspanne zu `_accumulated`, schreibt den Restpuffer und aktualisiert alle Zusammenfassungsspalten in `rides` mit einem einzigen `UPDATE`.

`sample(state)` wird vom MQTT-Thread bei jeder eingehenden Nachricht aufgerufen (10 Hz). Die Methode entscheidet selbst, ob der Punkt tatsächlich gespeichert wird – nur wenn seit dem letzten gespeicherten Punkt mindestens 200 ms vergangen sind (`SAMPLE_INTERVAL = 0.2`), wird ein neuer Trackpoint in den Puffer aufgenommen. Die Abtastrate für die Datenbank beträgt damit 5 Hz, was dem UI-Tick-Intervall des Dashboards entspricht.

Maximalwerte (Geschwindigkeit, Drehzahl, Schräglage) und die zurückgelegte Distanz werden laufend in Instanzvariablen akkumuliert, sodass `stop()` keine Nachberechnung über alle gespeicherten Punkte braucht. Die Streckenlänge wird mit der Haversine-Formel aus aufeinanderfolgenden GPS-Koordinaten summiert.

Da `sample()` aus dem MQTT-Thread und die Steuermethoden aus Dash-Callbacks aufgerufen werden, sichert ein `threading.Lock` jeden Zugriff auf den internen Zustand. SQLite wird mit `check_same_thread=False` geöffnet; die Synchronisation übernimmt der Lock des Recorders, nicht SQLite selbst.

6.3.3 Batch-Writes

Einzelne INSERT-Aufrufe bei jedem Messpunkt wären für die SD-Karte des Raspberry Pi ungünstig – jede SQLite-Transaktion erzwingt einen `fsync`, was auf SD-Karten deutlich langsamer ist als auf SSDs. Stattdessen sammelt der Recorder Punkte im RAM und schreibt sie gebündelt mit `executemany`:

```
FLUSH_EVERY = 25 # entspricht 5 Sekunden bei 5 Hz
```

```
def _flush_locked(self):
    if not self._buffer:
        return
    con = _connect()
    con.executemany(
        "INSERT INTO trackpoints "
        "(ride_id, t, lat, lng, speed, rpm, roll, pitch, temp) "
        "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)", self._buffer)
    con.commit()
    con.close()
    self._buffer = []
```

`_flush_locked()` wird automatisch aufgerufen, sobald der Puffer 25 Punkte enthält, sowie explizit bei `pause()` und `stop()`, damit beim Beenden einer Fahrt keine Punkte verloren gehen.

6.4 Autostart per systemd

Damit das Dashboard ohne manuelles Eingreifen nach dem Booten verfügbar ist, wird es als systemd-Dienst eingetragen. Mosquitto läuft bereits als systemd-Dienst und braucht nicht angepasst zu werden.

```
# /etc/systemd/system/motosense-dashboard.service
[Unit]
Description=MotoSense Dashboard
After=network-online.target mosquitto.service
Wants=network-online.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/motosense
ExecStart=/home/pi/motosense/venv/bin/python /home/pi/motosense/dashboard.py
```

```
Restart=on-failure
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

After=mosquitto.service stellt sicher, dass das Dashboard erst startet, nachdem der Broker betriebsbereit ist – andernfalls würde der MQTT-Thread beim Start sofort scheitern. ExecStart zeigt direkt auf den Python-Interpreter im venv, sodass kein vorheriges activate nötig ist. Restart=on-failure startet den Dienst bei einem Absturz automatisch neu, was im kopflosen Betrieb am Motorrad wichtig ist.

Aktivieren und starten:

```
sudo systemctl daemon-reload
sudo systemctl enable motosense-dashboard
sudo systemctl start motosense-dashboard
```

Mit `sudo journalctl -u motosense-dashboard -f` lassen sich die Live-Logs verfolgen, was bei der Fehlersuche ohne Bildschirm am Motorrad der einzige Diagnosepfad ist.

7 Web-Dashboard

Das Dashboard ist die einzige Benutzerschnittstelle von MotoSense. Es läuft als Webanwendung auf dem Raspberry Pi und wird auf dem Smartphone im Browser aufgerufen – keine App-Installation, keine Kabelverbindung.

7.1 Technologiewahl

Als Framework wird Plotly Dash eingesetzt. Dash erlaubt es, interaktive Webanwendungen vollständig in Python zu schreiben, ohne JavaScript anfassen zu müssen. Die Oberfläche wird serverseitig als HTML-Komponentenbaum definiert; Interaktionen laufen über deklarative Callbacks, die Ein- und Ausgaben von UI-Elementen verknüpfen.

Für die Kartenanzeige wird dash-leaflet verwendet, eine Dash-Anbindung an die JavaScript-Bibliothek Leaflet. Sie liefert eine interaktive Karte mit Marker und Streckenlinie direkt als Dash-Komponente, ohne dass eigener JavaScript-Code geschrieben werden muss. Als Kartenmaterial werden Kacheln von CartoCDN verwendet, die auch ohne API-Schlüssel abrufbar sind.

Gegenüber einer klassischen Flask-Anwendung mit manuell geschriebenem JavaScript-Frontend spart Dash erheblichen Aufwand: Gauges, Graphen und die Kartenkomponente sind fertige Bausteine, die nur parametrisiert werden müssen. Der Nachteil ist, dass jede UI-Aktualisierung einen HTTP-Roundtrip zum Server auslöst – bei 5 Hz und einem lokalen WLAN ist das kein Problem.

7.2 Architektur: MQTT-Thread und Dash-Callbacks

Dashboard und MQTT-Empfang laufen in getrennten Threads. Der MQTT-Thread verbindet sich beim Start mit dem Broker und empfängt Nachrichten in einer blockierenden `loop_forever()`-Schleife. Eingehende JSON-Pakete werden geparkt und in ein globales state-Dict geschrieben sowie in Ringpuffer (deque) für die Verlaufsgraphen eingetragen:

```
def on_message(client, userdata, msg):
    d = json.loads(msg.payload.decode())
    state.update(d)
    for k in hist:
        if d.get(k) is not None:
            hist[k].append(d[k])
    db.recorder.sample(state)
```

Das `subscribe()` wird bewusst nicht direkt nach `connect()` aufgerufen, sondern innerhalb des `on_connect`-Callbacks. Paho-MQTT ruft `on_connect` auch nach einem automatischen Reconnect auf – damit bleibt das Topic-Abonnement bei Verbindungsabbrüchen erhalten, ohne dass der Code dafür explizit eine Reconnect-Logik implementieren müsste.

Die Dash-Oberfläche liest `state` alle 200 ms über einen `dcc.Interval`-Trigger aus und zeichnet die Anzeigen neu. Schreibzugriffe kommen ausschließlich vom MQTT-Thread, Lesezugriffe ausschließlich von den Dash-Callbacks – der Python-GIL reicht für diese Konstellation als Synchronisationsmechanismus aus, ein expliziter Lock ist nicht nötig.

Ein wichtiges Dash-Verhalten betrifft Callbacks mit mehreren Outputs: Dash überspringt einen Callback vollständig, sobald auch nur ein einziges Ausgabeziel im aktuell gerenderten Tab nicht im DOM existiert. Ein gemeinsamer Callback für Home-Tab und Karten-Tab würde deshalb je nach aktivem Tab nie alle Ziele aktualisieren. Die Lösung ist, für jeden Tab einen eigenen Callback zu definieren, der ausschließlich Elemente des eigenen Tabs als Output hat.

7.3 Live-Ansicht

Die Live-Ansicht ist in drei Tabs unterteilt: Home, Karte und Fahrten.

Der Home-Tab zeigt die wichtigsten Kennwerte in Echtzeit. Die Geschwindigkeit wird als große Zahl im oberen Bereich hervorgehoben. Drehzahl und Temperatur erscheinen als kleinere Kennzahlkacheln daneben. Die aktuelle Schräglage wird als einzelner Winkelwert angezeigt, ergänzt durch zwei Kacheln für die maximale Schräglage links und rechts seit dem letzten Reset – ein Wert, der beim Motorradfahren praktisch interessant ist. Ein Button setzt die Maximalwerte zurück.

Der Karten-Tab zeigt die aktuelle GPS-Position als Marker auf einer Leaflet-Karte, die sich mit jedem Tick automatisch auf die neue Position bewegt. Darunter werden Koordinaten und Satellitenanzahl als Text ausgegeben.

Die Verlaufsgraphen im Fahrten-Tab werden über `plotly.graph_objs.Scatter` aus den Ringpuffern gezeichnet. Die deque mit `maxlen=150` hält automatisch nur die letzten 150 Werte – bei 5 Hz entspricht das 30 Sekunden Verlauf, ohne dass manuell alte Werte gelöscht werden müssen.

7.4 Fahrtenaufzeichnung und -historie

Die Aufnahmesteuerung im Home-Tab besteht aus drei Buttons (Beginnen, Pause, Beenden), die über Dash-Callbacks die entsprechenden Methoden des `RideRecorder` aus `db.py` aufrufen. Ein separater 1-Hz-Ticker aktualisiert die Statuszeile und den laufenden Timer unabhängig vom 5-Hz-Live-Ticker.

Der Fahrten-Tab zeigt eine Liste aller abgeschlossenen und laufenden Fahrten aus der Datenbank, neueste zuerst. Jeder Eintrag zeigt Name, Dauer, Streckenlänge, Höchstgeschwindigkeit und maximale Schräglage. Ein Tipp auf einen Eintrag öffnet die Detailansicht.

Die Detailansicht einer Fahrt enthält die gefahrene Route als Linie auf einer Karte, einen Scrubber zum Navigieren durch die aufgezeichneten Punkte sowie drei Graphen für Geschwindigkeit, Drehzahl und Schräglage über die Zeit. Der Scrubber bewegt einen Marker auf der Karte und zeigt die Messwerte des gewählten Zeitpunkts an. Die Trackpoints werden einmalig beim Öffnen der Detailansicht aus der Datenbank geladen und in einem clientseitigen `dcc.Store` abgelegt, damit Scrubber und Graphen-Callbacks ohne wiederholte Datenbankzugriffe arbeiten können.

Die Navigation zwischen Listen- und Detailansicht wird über Pattern-Matching-Callbacks abgewickelt: Jeder Fahrt-Button trägt eine ID der Form `{"type": "ride-item", "index": ride_id}`, auf die ein einzelner Callback mit `dash.ALL` reagiert. Dash feuert Pattern-Matching-Inputs

beim ersten Rendern mit `n_clicks=0` – diese Initial-Events werden explizit verworfen, da sie sonst beim Tab-Wechsel sofort in die Detailansicht springen würden.

7.5 Entwicklungshilfe: `fake_esp.py`

Während der Entwicklung war der ESP32 nicht immer verfügbar. `fake_esp.py` simuliert eine vollständige Motorradfahrt und publiziert die Daten mit 10 Hz auf dasselbe MQTT-Topic wie die echte Firmware – das Dashboard läuft damit ohne jede Codeänderung.

Die simulierte Fahrt ist in neun Phasen unterteilt (Leerlauf, Anfahren, Stadtverkehr, Kurvenstrecke usw.), die jeweils Zielgeschwindigkeit und Kurvigkeit vorgeben. Geschwindigkeit und Drehzahl werden träge ans Ziel angeglichen, die Schräglage ergibt sich aus Lenkeinschlag und Tempo, die GPS-Position wird anhand von Heading und Geschwindigkeit fortgeschrieben. Das Ergebnis sind plausibel korrelierende Sensorwerte, die auch die Fahrthistorie mit sinnvollen Daten befüllen.

Ein Zeitraffer-Faktor (SPEEDUP, Standard 5) verkürzt die simulierte Fahrtdauer für schnelle Iterationen. Mit `python fake_esp.py 1` once läuft die Fahrt einmalig in Echtzeit ab, was für Tests der Aufnahmesteuerung und der Datenbanklogik sinnvoll ist.

8 Inbetriebnahme

Dieses Kapitel beschreibt zwei Szenarien: den regulären Betrieb des fertig aufgebauten Systems sowie die Einrichtung auf einem neuen System von Grund auf. Ersteres richtet sich an den täglichen Einsatz am Motorrad, Letzteres dient als Anleitung, falls das System auf einem neuen Raspberry Pi oder mit einem neuen ESP32 neu aufgesetzt werden muss.

8.1 Regulärer Betrieb

8.1.1 Voraussetzungen

Bevor das System gestartet wird, sollten einige Punkte geprüft werden. ESP32 und Raspberry Pi müssen mit der externen Spannungsquelle verbunden sein. Die Drehzahlmesserschaltung wird separat über den Buck-Converter direkt vom Bordnetz des Motorrads versorgt – warum das so gelöst wurde, ist in Kapitel 11 erläutert. Alle Sensoren müssen korrekt verkabelt sein; die genaue Verkabelung ist in Kapitel 3 beschrieben.

Da das GPS-Modul eine freie Sicht zum Himmel benötigt, um Satellitensignale zu empfangen, sollte das Motorrad möglichst im Freien stehen. In geschlossenen Räumen oder in der Nähe von Gebäuden kann der Satellitenempfang eingeschränkt sein, was dazu führt, dass Geschwindigkeit und Position zu Beginn ungenau oder gar nicht angezeigt werden. Nach einigen Minuten im Freien ist der Fix in der Regel stabil.

8.1.2 Systemstart

Einer der praktischen Aspekte des Systems ist, dass nach dem Einschalten kein manueller Eingriff nötig ist. Der Raspberry Pi startet Mosquitto und das Dashboard automatisch über `systemd` – einen Dienstverwaltungsmechanismus von Linux, der festgelegte Programme beim Hochfahren automatisch ausführt. Nach etwa 20–30 Sekunden sind beide Dienste aktiv.

Auch der ESP32 agiert vollautomatisch: Er verbindet sich nach dem Einschalten selbstständig mit dem WLAN-Netz des Raspberry Pi (SSID: *MotoSense*) und beginnt sofort, Sensordaten über MQTT an den Pi zu senden.

8.1.3 Dashboard aufrufen

Das Smartphone muss sich im WLAN *MotoSense* befinden. Da dieses Netz keinen Internetzugang bietet, fragt iOS beim ersten Verbinden nach, ob die Verbindung trotzdem beibehalten werden soll –

dies muss bestätigt werden, da sonst automatisch auf Mobilfunk gewechselt wird und das Dashboard nicht erreichbar ist. Das Dashboard ist dann im Browser unter folgender Adresse abrufbar:

`http://192.168.4.1:8050`

Die IP-Adresse 192.168.4.1 ist die feste Adresse des Raspberry Pi in seinem eigenen WLAN-Netz. Da sie unveränderlich ist, muss sie nicht jedes Mal neu ermittelt werden.

8.1.4 Fahrtaufzeichnung

Im Home-Tab des Dashboards befinden sich drei Schaltflächen zur Steuerung der Aufzeichnung. Über *Beginnen* wird eine neue Fahrt gestartet, *Pause* unterbricht die Aufzeichnung vorübergehend und *Beenden* schließt die Fahrt ab und speichert sie dauerhaft in der Datenbank. Wichtig ist, dass die Aufnahme im Backend des Raspberry Pi läuft und nicht im Browser – sie wird also nicht unterbrochen, wenn der Browser-Tab geschlossen wird oder das Display des Smartphones ausschaltet. Abgeschlossene Fahrten sind anschließend im Fahrten-Tab einsehbar.

8.1.5 Systemstatus prüfen

Falls das Dashboard nach dem Einschalten nicht erreichbar ist, lässt sich per SSH auf dem Pi nachsehen, ob die Dienste korrekt gestartet sind. SSH ist ein Protokoll, das eine verschlüsselte Kommandozeilenverbindung zum Pi ermöglicht – vorausgesetzt, das Smartphone oder ein Laptop ist im selben Netz.

```
ssh pi@192.168.4.1
sudo systemctl status mosquitto
sudo systemctl status motosense-dashboard
```

Bei beiden Diensten sollte *active (running)* stehen. Falls ein Dienst abgestürzt ist, lassen sich die Fehlermeldungen über die Live-Logs einsehen:

```
sudo journalctl -u motosense-dashboard -f
```

8.2 Einrichtung auf einem neuen System

Dieser Abschnitt beschreibt Schritt für Schritt, wie das System auf einem frisch aufgesetzten Raspberry Pi und einem neuen ESP32 eingerichtet wird. Die Reihenfolge ist dabei wichtig: Zuerst wird der Pi vorbereitet, dann erst der ESP32 geflasht – da der ESP32 beim Start sofort versucht, eine Verbindung zum Pi aufzubauen, sollte der Pi zu diesem Zeitpunkt bereits als WLAN-Access-Point aktiv sein.

8.2.1 Voraussetzungen

Benötigt werden ein Raspberry Pi 4 mit Raspberry Pi OS Bookworm (64-Bit), ein ESP32-WROOM-32 DevKit sowie PlatformIO, das entweder als Erweiterung in VS Code oder als Kommandozeilenwerkzeug installiert sein kann. Der Projektordner muss auf dem Pi vorhanden sein – am einfachsten per USB-Stick oder über SCP (ein Dateiübertragungsprotokoll über SSH) übertragen.

8.2.2 Raspberry Pi einrichten

8.2.2.1 Mosquitto installieren

Mosquitto ist der MQTT-Broker, der alle Nachrichten zwischen ESP32 und Dashboard vermittelt. Er wird über den Paketmanager installiert und als Dienst eingerichtet, sodass er bei jedem Start automatisch läuft:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y mosquitto mosquitto-clients
```

```
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Standardmäßig akzeptiert Mosquitto nur Verbindungen vom Pi selbst. Damit auch der ESP32 Daten senden kann, muss eine kurze Konfigurationsdatei angelegt werden:

```
sudo nano /etc/mosquitto/conf.d/motosense.conf
```

Inhalt:

```
listener 1883
allow_anonymous true
```

Die erste Zeile öffnet den Port 1883 für eingehende Verbindungen, die zweite erlaubt Verbindungen ohne Passwort – für ein lokales, abgeschlossenes Netz am Motorrad ist das ausreichend. Danach Mosquitto neu starten:

```
sudo systemctl restart mosquitto
```

8.2.2.2 Python-Umgebung einrichten

Auf aktuellem Raspberry Pi OS ist das System-Python geschützt und darf nicht direkt mit zusätzlichen Paketen bestückt werden. Deshalb wird eine virtuelle Python-Umgebung (venv) angelegt – ein isolierter Bereich, in dem Pakete installiert werden können, ohne das System zu beeinflussen:

```
cd /home/pi/motosense
python3 -m venv venv
source venv/bin/activate
pip install paho-mqtt dash dash-leaflet plotly
```

Nach `source venv/bin/activate` erscheint (*venv*) vor dem Prompt, was zeigt, dass die virtuelle Umgebung aktiv ist.

8.2.2.3 Raspberry Pi als Access Point konfigurieren

Damit ESP32 und Smartphone sich direkt mit dem Pi verbinden können – ohne auf ein externes WLAN angewiesen zu sein – wird der Pi als WLAN-Access-Point eingerichtet. Das geschieht über NetworkManager, das unter Bookworm vorinstalliert ist:

```
sudo nmcli device wifi hotspot \
  ifname wlan0 \
  ssid MotoSense \
  password motosense123
```

Anschließend wird eine feste IP-Adresse vergeben und DHCP aktiviert, damit verbundene Geräte automatisch eine IP zugewiesen bekommen:

```
sudo nmcli connection modify hotspot \
  ipv4.addresses 192.168.4.1/24 \
  ipv4.method shared
sudo nmcli connection up hotspot
```

Damit der Access Point auch nach einem Neustart automatisch aktiv ist:

```
sudo nmcli connection modify hotspot \
  connection.autoconnect yes
```


8.2.2.4 systemd-Service einrichten

Damit das Dashboard beim Booten automatisch startet, wird ein systemd-Service angelegt. Dieser teilt Linux mit, wann und wie das Dashboard-Skript gestartet werden soll – in diesem Fall nach dem Hochfahren des Netzwerks und nach dem Start von Mosquitto:

```
sudo nano /etc/systemd/system/motosense-dashboard.service
```

Inhalt:

```
[Unit]
Description=MotoSense Dashboard
After=network-online.target mosquitto.service
Wants=network-online.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/motosense
ExecStart=/home/pi/motosense/venv/bin/python /home/pi/motosense/dashboard.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Restart=on-failure sorgt dafür, dass das Dashboard bei einem Absturz automatisch neu gestartet wird – praktisch, da am Motorrad kein Bildschirm angeschlossen ist. Aktivieren und starten:

```
sudo systemctl daemon-reload
sudo systemctl enable motosense-dashboard
sudo systemctl start motosense-dashboard
```

8.2.3 ESP32 flashen

Den Projektordner in PlatformIO öffnen. Die Datei `platformio.ini` enthält bereits alle benötigten Bibliotheken – PlatformIO lädt sie beim ersten Build automatisch aus dem Internet nach, sofern eine Verbindung besteht.

In `main.cpp` müssen die WLAN-Zugangsdaten auf die des Pi-Access-Points gesetzt sein:

```
const char* WIFI_SSID = "MotoSense";
const char* WIFI_PASS = "motosense123";
const char* MQTT_HOST = "192.168.4.1";
```

Anschließend den ESP32 per USB anschließen und die Firmware übertragen:

```
pio run --target upload
```

PlatformIO erkennt das Board automatisch. Falls der Upload fehlschlägt, kann es helfen, beim Erscheinen von *Connecting...* den BOOT-Button auf dem Board gedrückt zu halten – einige ESP32-Boards benötigen das beim Flashen.

Nach dem Upload kann über den Serial Monitor kontrolliert werden, ob der ESP32 erfolgreich eine WLAN-Verbindung aufbaut und Daten sendet:

```
pio device monitor --baud 115200
```

Im Log sollte nach wenigen Sekunden *MQTT verbunden* erscheinen und danach im Sekundentakt die gesendeten JSON-Pakete.

8.2.4 Elektronik und Mechanik

Der Aufbau der Sensorschaltungen – Verkabelung der einzelnen Sensoren, Signalkonditionierung für den Drehzahlsensor und Auslegung der Spannungsversorgung – ist vollständig in Kapitel 3 beschrieben. Die mechanischen Halterungen, deren CAD-Modelle und die Montage am Fahrzeug sind in Kapitel 4 dokumentiert. Die zugehörigen CAD-Dateien liegen im Projektordner unter cad/.

9 Aufwandsvergleich und Arbeitsprotokoll

9.1 Geplanter vs. tatsächlicher Aufwand

Die folgende Tabelle stellt den bei der Projektvorstellung geschätzten Aufwand dem tatsächlich geleisteten gegenüber. Die Werte aus dem Zwischenstandsbericht wurden um die in der Abschlussphase angefallenen Stunden ergänzt.

Bereich / Aufgabe	Geplant (Std)	Ist (Std)	Differenz	Anmerkung
Elektronik				
Sensorauswahl	5	12	+7	Aufwändiger als erwartet – PT100/PT1000-Verwechslung, Recherche MAX31865-Klone
Microcontroller-Auswahl	5	3	–2	ESP32 war schnell gesetzt
Verkabelung der Sensoren	15	12	–3	Abgeschlossen
Betriebsspannung anpassen	5	8	+3	Bordnetz-Instabilität erforderte externe Versorgung
Gesamtverdrahtung	15	14	–1	RPM-Kreis galvanisch getrennt
Summe Elektronik	45	49	+4	
Mechanik				
Planung mechanische Teile	10	12	+2	Abgeschlossen
Anfertigung mechanische Teile	2	0,5	–1,5	Kaum manuelle Fertigung nötig
3D-Druck	20	18	–2	Abgeschlossen
Modellierung (CAD)	15	21	+6	Mehr Iterationen als geplant

Bereich / Aufgabe	Geplant (Std)	Ist (Std)	Differenz	Anmerkung
Montage	6	3	-3	Kein fester Einbau ins Motorrad – deutlich weniger Aufwand
Summe Mechanik	53	54,5	+1,5	
Informatik				
Softwareplanung	10	10	0	Abgeschlossen
Sensoranbindung ESP32	15	18	+3	RPM-ISR-Bug, PC817-Pullup-Problem, PT100-Korrektur
Kommunikation ESP32 – Raspberry Pi	5	6	+1	MQTT inkl. Reconnect-Logik
Datenbank	10	10	0	SQLite + RideRecorder abgeschlossen
Web-App (Dashboard)	10	14	+4	Tab-Callback-Problem in Dash 4, Kartenintegration
Softwaretests	5	6	+1	fake_esp.py als vollständiger Simulator
Dokumentation	10	12	+2	Laufend erstellt
Summe Informatik	65	76	+11	
Gesamtaufwand	163	179,5	+16,5	

Tabelle 5: Aufwandsvergleich: Planung vs. tatsächlicher Aufwand

9.1.1 Elektronik

Der Elektronikbereich erwies sich insgesamt als anspruchsvoller als in der Zielsetzung angenommen. Den größten Mehraufwand verursachte die Sensorauswahl: Das eingesetzte MAX31865-Board stellte sich erst nach längerer Fehlersuche als PT100-Klon heraus – der Aufdruck „431“ auf dem SMD-Widerstand ($430\ \Omega$ statt der für PT1000 erwarteten $4300\ \Omega$) war der entscheidende Hinweis. Bis zur korrekten Parametrierung wurden Temperaturen mit einem systematischen Fehler von mehreren hundert Grad ausgegeben. Dieser Vorfall ist im Kapitel „Probleme und Lösungen“ ausführlich dokumentiert.

Ein weiterer ungeplanter Mehraufwand entstand durch die Instabilität der Bordnetz-Spannung am Versuchsfahrzeug. Spannungsspitzen beim Starten und Betrieb des Motors führten wiederholt zu Ausfällen des Mikrocontrollers, sodass ESP32 und Raspberry Pi schließlich auf eine externe Powerbank umgestellt wurden. Nur der RPM-Signalfad verbleibt am Bordnetz, galvanisch durch einen Optokoppler getrennt. Auch dieser Punkt wird in „Probleme und Lösungen“ behandelt.

9.1.2 Mechanik

Der mechanische Aufwand fiel deutlich geringer aus als ursprünglich geplant. Da auf einen festen Einbau der Elektronik ins Motorrad verzichtet wurde und die Komponenten stattdessen in einem abnehmbaren Gehäuse untergebracht sind, entfielen aufwändige Montagearbeiten am Fahrzeug weitgehend. Der Mehraufwand bei der CAD-Modellierung (+6 Std) erklärt sich durch mehrere Iterationen bei der Gehäusekonstruktion, bis Passform und Kabelführung zufriedenstellend waren.

9.1.3 Informatik

Die Softwareentwicklung verlief insgesamt nah an der ursprünglichen Planung, mit moderatem Mehraufwand in einzelnen Bereichen. Bei der Firmware kostete die Drehzahlmessung die meiste Zeit: Ein Initialisierungsfehler in der ISR (lastPulseUs = 0 beim Start) produzierte beim ersten Puls ein massives, unrealistisches Intervall, das den Plausibilitätsfilter dauerhaft blockierte und die Drehzahl bei null hielt. Die Ursache war nicht sofort offensichtlich, da die Schaltung elektrisch korrekt aufgebaut war. Hinzu kam ein doppelter Pull-up-Widerstand am PC817-Breakout-Board, der den Ausgang daran hinderte, sicher auf Low zu ziehen.

Im Dashboard bereitete das Callback-Modell von Dash 4 unerwartete Probleme: Callbacks werden still übergangen, sobald auch nur ein einziges Ausgabeziel im aktuell sichtbaren Tab nicht im DOM vorhanden ist. Die Lösung war eine strikte Trennung der Callbacks nach Tab. Der Simulator fake_esp.py wurde zu einem vollständigen Fahrtsimulator mit phasenbasiertem Geschwindigkeitsprofil, korrelierter Schräglage und realistischer GPS-Spurberechnung ausgebaut, was den Testaufwand für Dashboard und Datenbank erheblich vereinfachte.

9.2 Stundenliste

Datum	Kategorie	Std.	Tätigkeit
07.04.2026	Elektronik	2	Sensorrecherche: GPS, IMU, Temperatur, RPM
08.04.2026	Elektronik	3	Mikrocontroller-Auswahl: ESP32-WROOM-32
09.04.2026	Elektronik	3.5	Sensorauswahl abgeschlossen, Komponentenliste erstellt
10.04.2026	Informatik	3	PlatformIO eingerichtet, Bibliotheken installiert
11.04.2026	Mechanik	3	Mechanische Konzeption: Gehäuse, Anforderungen, Skizzen
12.04.2026	Mechanik	3	CAD-Modellierung Sensor-Gehäuse in Fusion 360
13.04.2026	Mechanik	2.5	CAD-Modellierung Sensor-Gehäuse weitergeführt
14.04.2026	Mechanik	3	CAD-Modellierung abgeschlossen, STL exportiert
16.04.2026	Informatik	2	Git-Repo angelegt, Projektstruktur und README erstellt
17.04.2026	Elektronik	3	Drehzahlschaltung entworfen: Pickup, Schmitt-Trigger, Optokoppler
18.04.2026	Elektronik	3.5	74HC14 auf Breadboard aufgebaut, Schaltplan erstellt
19.04.2026	Mechanik	3	RPM-Gehäuse modelliert und gedruckt
20.04.2026	Elektronik	2.5	PC817-Optokoppler-Stufe aufgebaut, Flanken geprüft
21.04.2026	Elektronik	2	GPS-Modul an UART2 verkabelt und getestet
22.04.2026	Elektronik	2.5	IMU an I2C angeschlossen, I2C-Scanner, Kalibrierung
23.04.2026	Elektronik	3	MAX31865 an VSPI verkabelt, Fault 0x80 diagnostiziert
24.04.2026	Elektronik	3.5	MAX31865-Diagnose: SPI ok, Jumper, SMD-Widerstand analysiert
25.04.2026	Elektronik	2	PT100/PT1000-Verwechslung aufgelöst, Code angepasst

Datum	Kategorie	Std.	Tätigkeit
26.04.2026	Mechanik	4	Stromverteiler-Gehäuse und Lenkerhalterung modelliert
27.04.2026	Mechanik	3.5	3D-Druck weiterer Teile, Passgenauigkeit geprüft
28.04.2026	Elektronik	2.5	LM2596-Buck-Converter kalibriert, Stromverteiler gebaut
29.04.2026	Informatik	2.5	Raspberry Pi eingerichtet, Mosquitto konfiguriert
30.04.2026	Informatik	2	MQTT-Funktionstest, Python-venv, paho-mqtt installiert
02.05.2026	Elektronik	3	Pickup-Spule am Zündkabel montiert, RPM-Schaltung gelötet
03.05.2026	Informatik	2.5	GPS+IMU-Sketch kombiniert, Serial-Ausgabe
04.05.2026	Elektronik	3.5	Bordnetz-Instabilität festgestellt, ESP32/Pi auf externe Versorgung
05.05.2026	Elektronik	2	Optokoppler-Ausgang analysiert: doppelter Pull-up als Ursache
06.05.2026	Elektronik	2.5	Externen Pull-up entfernt, Signal korrekt 0–3,3 V
07.05.2026	Informatik	3	ISR-Bug behoben: lastPulseUs in setup() initialisiert
08.05.2026	Informatik	2	RPM-Plausibilitätsfilter verfeinert
09.05.2026	Informatik	2.5	Temperatur-Plausibilitätscheck eingebaut
10.05.2026	Mechanik	3	3D-Druck-Teile montiert, Pi-Halterung am Rahmen
11.05.2026	Elektronik	2.5	Temperaturkalibrierung: Offset –10 °C eingestellt
12.05.2026	Informatik	2	IMU-Standstill-Erkennung implementiert
13.05.2026	Informatik	3	GPS+IMU+Temp+RPM zusammengeführt, JSON-Payload erstellt
14.05.2026	Informatik	2.5	ESP32-Firmware: WiFi + MQTT-Publishing eingebaut
17.05.2026	Informatik	4	Alle Sensoren zusammengeführt, Firmware finalisiert
18.05.2026	Informatik	2.5	fake_esp.py Simulator geschrieben
22.05.2026	Elektronik	2	RPM Debugging am Fahrzeug
23.05.2026	Mechanik	4.5	Endmontage Fahrzeug, Kabelstrang gebündelt
24.05.2026	Elektronik	2	Abschlusstest aller Sensoren im Fahrzeug
25.05.2026	Mechanik	2	Abschließende mechanische Korrekturen
27.05.2026	Informatik	2.5	fake_esp.py auf 9-Phasen-Fahrsimulation umgestellt
01.06.2026	Informatik	3	Mosquitto-Konfiguration, WiFi und MQTT-Publishing finalisiert
03.06.2026	Informatik	3.5	Erstes Dash-Dashboard: Gauges, Karte, Temperaturverlauf
06.06.2026	Informatik	4.5	db.py: SQLite, RideRecorder, Haversine-Distanz
09.06.2026	Informatik	4	Dashboard-Redesign: Tab-Navigation, Fahrten-Tab
11.06.2026	Informatik	3	Dash-Callback-Bug, MQTT-Subscribe, Dashboard finalisiert
14.06.2026	Informatik	3	Raspberry Pi als WLAN-Access-Point konfiguriert, systemd-Service
16.06.2026	Informatik	3.5	Probefahrt, Dashboard und Fahrtaufzeichnung getestet
19.06.2026	Informatik	2	Kalibrierungsoffset, Stillstandsschwellwerte und RPM-Parameter angepasst

Datum	Kategorie	Std.	Tätigkeit
02.06.2026	Dokumentation	3.5	Dokumentation: Kapitel 1 Einleitung, Kapitel 2 Systemarchitektur
05.06.2026	Dokumentation	3	Dokumentation: Kapitel 3.1–3.3 GPS
08.06.2026	Dokumentation	3	Dokumentation: Kapitel 3.4 IMU, 3.5.1 Spannungsversorgung
11.06.2026	Dokumentation	3.5	Dokumentation: Kapitel 3.5.2 Buck-Converter, 3.6 MAX31865
14.06.2026	Dokumentation	3	Dokumentation: Kapitel 3.7 Drehzahlmessung
16.06.2026	Dokumentation	3.5	Dokumentation: Kapitel 4 Mechanik
18.06.2026	Dokumentation	4	Dokumentation: Kapitel 5 Firmware, Kapitel 6 Dashboard
20.06.2026	Dokumentation	3	Dokumentation: Kapitel 7 Raspberry Pi Setup
22.06.2026	Dokumentation	3.5	Dokumentation: Kapitel 8 Probleme und Lösungen, Endredaktion
Gesamt		179.5	

10 Commit-Historie

Datum	Commit-Nachricht
16.04.2026	Initial commit
16.04.2026	created basic folder structure, changed README
16.04.2026	changed README
16.04.2026	Erste CAD-Modelle esp32 Sensor gehäuse
18.04.2026	Schaltplan Drehzahlmesserschaltung (Schmitt-Trigger, Optokoppler)
19.04.2026	RPM-Gehäuse
26.04.2026	Stromverteiler-Gehäuse modelliert
02.05.2026	GPS-Modul (NEO-6M) an UART2 anschließen und testen
05.05.2026	MPU-6050 an I2C anschließen
10.05.2026	RPM-Interrupt an GPIO27, Plausibilitätsfilter implementieren
13.05.2026	lastPulseUs in setup() initialisieren, ISR-Bug beheben
17.05.2026	Alle Sensoren zusammenführen, JSON-Payload auf motosense/data
22.05.2026	RPM Debugging
27.05.2026	fake_esp.py auf 9-Phasen-Fahrsimulation umstellen
01.06.2026	Mosquitto-Konfiguration, WiFi und MQTT-Publishing am ESP32
03.06.2026	Erstes Dash-Dashboard mit Gauges, Karte, Temperaturverlauf
06.06.2026	db.py mit SQLite, RideRecorder und Haversine-Distanz
11.06.2026	Dash-Callbacks nach Tab aufteilen, MQTT-Subscribe in on_connect, Dashboard finished
14.06.2026	Raspberry Pi als Access-Point konfigurieren, systemd-Service einrichten
19.06.2026	Kalibrierungsoffset, Stillstandsschwellwerte und RPM-Parameter anpassen
22.06.2026	Dokumentation und Video

11 Probleme und Lösungen

Während der Entwicklung von MotoSense traten an mehreren Stellen Probleme auf, die nicht durch einfaches Nachschlagen lösbar waren, sondern gezieltes Debuggen und in einigen Fällen einen grundlegenden Umbau erforderten. Die folgenden Abschnitte dokumentieren die wesentlichen Schwierigkeiten chronologisch nach ihrem Auftreten, beschreiben den jeweiligen Diagnoseprozess und erläutern die gewählte Lösung samt Begründung.

11.1 ESP32-Upload-Fehler: TX-Pfad durch GPS-Leitung blockiert

Beim ersten Flashversuch nach dem Anschließen des GPS-Moduls erschien im PlatformIO-Terminal die Meldung „*Download mode successfully detected, but getting no sync reply: the serial TX path seems to be down*“. Der ESP32 wurde vom Flasher zwar erkannt – der BOOT-Modus wurde korrekt ausgelöst – aber die eigentliche Datenübertragung schlug reproduzierbar fehl. Verschiedene USB-Kabel, unterschiedliche Übertragungsgeschwindigkeiten und das manuelle Halten des BOOT-Buttons änderten nichts an dem Fehler.

Die Ursache lag in einer falsch angeschlossenen GPS-Leitung. Der TX-Pin des NEO-6M-Moduls war versehentlich an GPIO1 des ESP32 geführt worden statt an GPIO16. GPIO1 ist jedoch der TX0-Pin der ersten Hardware-UART, über die auch die USB-UART-Brücke des DevKits läuft. Das GPS-Modul

hielt damit dauerhaft Daten auf die Leitung, blockierte den Sendepfad des Flashers und verhinderte die Synchronisation.

Der Fix war nach der Diagnose trivial: GPS-TX auf GPIO16 (RX2) umstecken, GPIO1 und GPIO3 beim Upload frei lassen. Als allgemeine Regel gilt: An GPIO0, GPIO1 und GPIO3 darf beim Flashen nichts angeschlossen sein, da diese Pins zur Steuerung des Boot-Modus und zur Upload-UART gehören.

11.2 GPS-Phantomgeschwindigkeit bei Stillstand

Im frühen Testbetrieb – mit dem ESP32 auf dem Schreibtisch, GPS-Modul ohne freie Himmelssicht – zeigte das Dashboard im Stillstand konstant Geschwindigkeitswerte zwischen 2 und 5 km/h. Die Werte schwankten leicht und waren nicht konstant null, obwohl das System offensichtlich stand.

Die Ursache liegt im Verhalten des NEO-6M bei schlechtem Empfang. Mit wenigen oder schwachen Satelliten ist die Positionsberechnung ungenau, der berechnete Aufenthaltsort springt von Messung zu Messung um wenige Meter. Die fahrzeuginterne Geschwindigkeitsberechnung des GPS-Chips wertet diese Positionssprünge als Bewegung und liefert entsprechende Geschwindigkeitswerte – obwohl das Modul still liegt. Dieses Verhalten ist dokumentiert und tritt bei allen Low-Cost-GPS-Modulen ohne echten Doppler-basierten Geschwindigkeitssensor auf.

Als Lösung wurde eine IMU-gestützte Stillstandserkennung in die Firmware integriert. Der MPU-6050 liefert kontinuierlich Beschleunigungs- und Gyroskopdaten. Liegt der Betrag des Beschleunigungsvektors – bereinigt um den Schwerkraftanteil – unterhalb eines empirisch bestimmten Schwellwerts und sind gleichzeitig alle drei Gyroskopraten nahe null, wird die GPS-Geschwindigkeit im ausgesendeten JSON auf 0 gesetzt. Der Filter ist so dimensioniert, dass normales Fahren ihn nicht auslöst, ein stehendes Motorrad ihn aber sicher aktiviert. Damit verschwinden die Phantomwerte zuverlässig, ohne den regulären Fahrbetrieb zu beeinflussen.

11.3 Dash-Callback-Ausfall bei fehlendem DOM-Element

Nach dem Umbau des Dashboards von einer einzelnen Seite auf eine Tab-Struktur mit drei Tabs – Home, Karte, Fahrten – aktualisierte die Live-Anzeige einzelne Elemente nicht mehr. Konkret blieben Geschwindigkeit und Drehzahl auf dem Home-Tab eingefroren, sobald zuvor kurz auf den Karten-Tab gewechselt und dann zurückgekehrt worden war. Im Browser-Konsolenlog erschienen keine Fehlermeldungen, Dash selbst meldete nichts Auffälliges.

Die Ursache ist ein Verhalten von Dash 4, das in der Dokumentation nur beiläufig erwähnt wird: Enthält ein Callback auch nur ein einziges Output-Ziel, das im aktuell gerenderten DOM nicht vorhanden ist, überspringt Dash den gesamten Callback – stillschweigend, ohne Exception. Da beim Tab-Wechsel jeweils nur der Inhalt des aktiven Tabs im DOM liegt, fehlten nach einem Wechsel zum Karten-Tab alle Home-spezifischen Elemente. Ein gemeinsamer Callback, der gleichzeitig auf Home- und Karten-Elemente schreibt, wurde von Dash komplett verworfen, sobald eine der beiden Seiten nicht aktiv war.

Die Lösung bestand darin, den ursprünglichen gemeinsamen Update-Callback vollständig aufzuteilen: ein Callback ausschließlich für die Home-Tab-Elemente, ein separater für die Kartenelemente. Jeder Callback schreibt nur auf Ziele, die garantiert gleichzeitig im DOM existieren. Seitdem laufen beide Ansichten unabhängig voneinander korrekt.

11.4 MQTT-Subscription nach Reconnect verloren

Nach einem simulierten Verbindungsabbruch – Mosquitto-Dienst kurz neugestartet – empfing das Dashboard anschließend keine Nachrichten mehr, obwohl der MQTT-Client die Verbindung laut Log wiederhergestellt hatte. Das Verbindungs-Wiederaufbauen funktionierte, aber die Daten blieben aus.

Das Problem lag darin, dass `subscribe()` direkt nach `connect()` aufgerufen wurde – also einmalig beim Programmstart. Paho-MQTT stellt nach einem Reconnect die TCP-Verbindung automatisch wieder her und ruft dabei den `on_connect`-Callback auf, aber das Topic-Abonnement aus der ursprünglichen Session wird nicht automatisch erneuert. Da `subscribe()` außerhalb von `on_connect` stand, wurde es beim Reconnect nicht erneut ausgeführt, und der Client lauschte auf kein Topic mehr.

Die Lösung ist konzeptuell einfach, aber nicht offensichtlich: `subscribe()` muss ausschließlich innerhalb des `on_connect`-Callbacks stehen. Paho ruft diesen bei jedem erfolgreichen Verbindungsaufbau auf, egal ob beim ersten Start oder nach einem Reconnect. Das Abonnement wird damit automatisch bei jeder Verbindung erneuert, ohne dass dafür eigene Reconnect-Logik geschrieben werden muss.

11.5 RPM-Signalqualität: Debugging der Schaltung und galvanische Trennung

Die Drehzahlmessung bereitete die meisten Schwierigkeiten im gesamten Projekt und durchlief mehrere Debugging-Stufen, bevor sie zuverlässig funktionierte.

11.5.1 Erste Tests: Rohdaten am GPIO

Als erster Schritt wurde ein minimaler Testsketch eingesetzt, der lediglich Pulse an GPIO27 zählt und das letzte gemessene Intervall ausgibt. Damit ließ sich unabhängig von der restlichen Firmware feststellen, ob überhaupt Pulse ankamen und wie sauber sie waren. Der Signalgenerator – ein zweiter ESP32, der ein Rechtecksignal mit einstellbarer Frequenz erzeugte – lieferte dabei das Testsignal, um die Schaltung ohne Motorrad prüfen zu können.

Die Rohdaten zeigten zunächst ein grundlegendes Problem: Für jeden simulierten Puls wurden zwei bis vier Interrupts ausgelöst. Der Zähler wuchs deutlich schneller als die Eingangsfrequenz, und die berechneten Intervalle schwankten stark. Das Signal war zu diesem Zeitpunkt noch nicht durch den Schmitt-Trigger geführt worden – ein direkt angeschlossenes RC-Glied reichte nicht aus, um das Prellen zu unterdrücken.

11.5.2 Schmitt-Trigger (74HC14): Mehrfachtrigger beseitigt

Nach dem Einschleifen des 74HC14 zwischen Signaleingang und GPIO27 verschwanden die Mehrfachtrigger. Der Schmitt-Trigger formt die langsamen, verrauschten Flanken des induktiv abgenommenen Signals in scharfe digitale Übergänge um. Die Hysterese des 74HC14 – typisch 0,8 V bei 3,3-V-Betrieb – verhindert, dass Signalrauschen nahe der Schaltschwelle mehrfach auslöst. Der Rohdaten-Testsketch bestätigte anschließend ein sauberes Eins-zu-eins-Verhältnis zwischen Eingangspulsen und gezählten Interrupts.

11.5.3 PC817-Optokoppler: Pegel zu hoch für ESP32-LOW

Der nächste Schritt war das Einschleifen des PC817-Optokopplers für die galvanische Trennung. Danach zählte der Testsketch gar keine Pulse mehr, obwohl das Signal am Eingang des Optokopplers mit dem Oszilloskop sauber aussah. GPIO27 zeigte dauerhaft HIGH, unabhängig vom Eingangssignal.

Die Fehlersuche begann mit dem Messen der Spannung am Kollektor-Ausgang des PC817. Im durchgeschalteten Zustand – also wenn die Optokoppler-LED leuchtet und der Phototransistor leitend sein sollte – fiel die Spannung nur auf etwa 1,4 V statt auf unter 0,8 V. Der ESP32 wertet Spannungen über ca. 0,8 V als HIGH, weshalb er den durchgeschalteten Zustand nicht als LOW erkannte und kein FALLING-Interrupt ausgelöst wurde.

Die Ursache war ein doppelter Pull-up-Widerstand: Das verwendete PC817-Breakout- Board trägt bereits einen eigenen 10-k Ω -Pull-up-Widerstand zwischen dem Kollektor-Ausgang und VCC. Zusätzlich war ein externer 10-k Ω -Pull-up nach der Schaltungsbeschreibung bestückt worden. Beide Widerstände lagen parallel am Ausgang, was einem effektiven Pull-up von 5 k Ω entspricht. Ein schwächerer Pull-up bedeutet, dass der Phototransistor einen niedrigeren Widerstand aufweisen müsste, um die Ausgangsspannung auf LOW zu ziehen – der PC817 schafft das mit 5 k Ω nicht mehr sicher, insbesondere bei den verwendeten LED-Strömen.

Die Lösung war das Entfernen des externen Pull-up-Widerstands. Mit nur noch dem 10-k Ω -Widerstand des Breakout-Boards fiel die Ausgangsspannung im durchgeschalteten Zustand auf unter 0,3 V, was der ESP32 sicher als LOW erkennt. Danach arbeitete der Optokoppler korrekt, und der Testsketch zählte Pulse im exakten Verhältnis zur Eingangsfrequenz.

11.5.4 Firmware: ISR-Initialisierung und Plausibilitätsfilter

Nach der Lösung der Hardwareprobleme trat ein weiterer Fehler in der Firmware auf: Beim Start zeigte das Dashboard kurzzeitig eine unrealistisch hohe Drehzahl – mehrere zehntausend 1/min – für wenige Sekunden, bevor der Wert auf null zurückfiel und sich nicht mehr erholte.

Die Ursache war die Initialisierung von `lastPulseUs` mit dem Wert 0. Der erste eintreffende Puls berechnete damit ein Intervall von `micros() - 0`, also die gesamte Zeit seit dem Systemstart – typischerweise mehrere hundert Millisekunden. Dieser riesige Wert wurde als `lastGoodIntervalUs` gespeichert. Das nächste reguläre Pulsintervall – beispielsweise 50 ms bei 1200 1/min – war deutlich kürzer als 60 % von mehreren hundert Millisekunden, weshalb der Plausibilitätsfilter es als unplausibel verwarf. Da jeder folgende Puls dasselbe Schicksal erlitt, blieb die gemessene Drehzahl dauerhaft auf null.

Die Lösung war, `lastPulseUs` in `setup()` unmittelbar vor `attachInterrupt()` mit `micros()` zu initialisieren statt mit 0. Damit berechnet der erste eintreffende Puls ein reales Intervall seit dem letzten Systemereignis, und der Plausibilitätsfilter arbeitet von Beginn an korrekt.

11.6 PT100/PT1000-Verwechslung

Das eingesetzte MAX31865-Breakout-Board wurde über den Händler als kompatibel für PT1000-Sonden beschrieben. Dementsprechend wurden in der Firmware `RREF = 4300.0` und `RNOMINAL = 1000.0` gesetzt. Die ausgelesenen Temperaturen wichen jedoch systematisch um mehrere Dutzend Grad vom erwarteten Wert ab. Bei Raumtemperatur zeigte der Sensor Werte um 200 °C, beim Halten eines Fingers an die Sonde änderte sich der Wert kaum.

Der erste Diagnoseschritt war das Auslesen des Rohwiderstands der Sonde – also ohne Temperaturumrechnung, direkt aus dem ADC-Rohwert. Bei Raumtemperatur ergab sich ein Widerstand von ca. 110 Ω . Ein PT1000 hätte bei 26 °C etwa 1100 Ω , ein PT100 hingegen etwa 110 Ω . Damit war klar: Die Sonde ist ein PT100, kein PT1000.

Im zweiten Schritt wurde der Referenzwiderstand auf dem Board selbst gemessen und mit der Bauteilbeschriftung abgeglichen. Der SMD-Widerstand neben dem MAX31865-Chip trägt den Aufdruck „431“. Im E96-Normwert-System entspricht „431“ dem Wert 430 Ω – nicht 4300 Ω wie beim Adafruit-Original-PT1000-Board, das den Aufdruck „4301“ trägt. Das Board ist damit eine PT100-Platine mit 430- Ω -Referenzwiderstand, keine PT1000-Platine.

Die Korrektur bestand aus zwei Zeilen Firmware: `RREF = 430.0` und `RNOMINAL = 100.0`. Danach stimmten die Messwerte mit einem Referenzthermometer auf ± 1 °C überein.

11.7 Bordnetz-Instabilität beim Motorradstart

Beim ersten Versuch, ESP32 und Raspberry Pi aus dem 12-V-Bordnetz der Bandit 1200 über LM2596-Schaltregler zu versorgen, stürzten beide Systeme beim Betätigen des Anlassers reproduzierbar ab. Der ESP32 startete neu, der Raspberry Pi fuhr manchmal vollständig hoch, manchmal blieb er in einem inkonsistenten Zustand und musste manuell neugestartet werden.

Die Ursache liegt im Verhalten des Bordnetzes beim Anlassen. Ein Motorrad- Anlasser zieht je nach Hubraum und Temperatur Ströme von 50 bis über 100 A für die Dauer des Startvorgangs. Dieser hohe Strombedarf führt im Bordnetz zu kurzzeitigen Spannungseinbrüchen auf unter 9 V sowie zu induzierten Spannungsspitzen beim Abschalten des Anlassers. LM2596-Schaltregler haben eine begrenzte Eingangskapazität und reagieren auf solche Transienten nicht schnell genug: Fällt die Eingangsspannung kurz unter die Mindestversorgung, bricht auch die Ausgangsspannung ein, und der ESP32 unterschreitet seine Mindestbetriebsspannung von 3,0 V.

Als praktische Lösung für den Versuchsbetrieb wurden ESP32 und Raspberry Pi vollständig vom Bordnetz getrennt und auf eine externe USB-Powerbank umgestellt. Die Powerbank liefert eine stabile, geregelte Spannung unabhängig vom Zustand der Motorradbatterie und ist von Anlasservorgängen völlig unberührt. Nur der RPM-Signalkreis verbleibt am Bordnetz, da er die Zündimpulse direkt vom Zündkabel abnehmen muss. Der Optokoppler übernimmt dabei die vollständige galvanische Trennung zwischen der Bordnetz-Seite und dem ESP32.

Für eine spätere Integration einer direkten Bordnetzversorgung wären mindestens ein TVS-Diodenschutz am Eingang, ein Verpolschutzmosfet und ein Bulk-Kondensator von mindestens 470 µF erforderlich, um die transienten Belastungen beim Anlassen sicher abzufangen. Außerdem wäre ein Schaltregler mit schnellerer Transient-Response oder ein dediziertes KFZ-Versorgungsmodul zu bevorzugen.

11.8 Wechsel der Spannungsversorgung: Bordnetz auf Powerbank

Der ursprüngliche Entwurf sah eine vollständige Bordnetzversorgung vor, da dies die eleganteste Lösung wäre: System einschalten, Motor starten, losfahren. Nach den Erfahrungen mit den Bordnetz-Instabilitäten beim Anlassen (Abschnitt 10.8) wurde dieser Ansatz für den Versuchsbetrieb verworfen und durch eine Powerbank-Versorgung ersetzt.

Die Entscheidung für eine Powerbank hat neben der erhöhten Stabilität weitere praktische Vorteile: Der Aufbau ist unabhängig vom Ladezustand der Motorradbatterie, er benötigt keinen dauerhaften Eingriff in die Fahrzeugelektrik, und das gesamte System lässt sich vom Motorrad abnehmen und am Schreibtisch weiter entwickeln, ohne die Motorradbatterie zu belasten.

Der Nachteil ist die begrenzte Kapazität. Eine 10.000-mAh-Powerbank versorgt ESP32 und Raspberry Pi 4 gemeinsam für ca. 3–4 Stunden, bevor sie aufgeladen werden muss. Für die geplante Nutzung – Tests und Entwicklungsfahrten – ist das ausreichend.

Die Trennung der Versorgungsdomänen macht die galvanische Trennung am RPM-Kreis nicht optional, sondern zwingend: Die Primärseite des Optokopplers (Pickup, Schmitt-Trigger, LED) liegt auf Bordnetz-Potential mit dessen Massebezug; die Sekundärseite (Phototransistor, Pull-up, GPIO27) liegt auf Powerbank-Potential mit dem ESP32-GND als Referenz. Beide Massen dürfen nicht verbunden werden – eine Verbindung würde den Optokoppler kurzschließen und Bordnetz-Störungen direkt in den ESP32 einspeisen.

12 Fazit

MotoSense hat sein Ziel erreicht: Ein funktionsfähiges, nachrüstbares Telemetriesystem für die Suzuki Bandit 1200, das Drehzahl, Öltemperatur, Schräglage und GPS-Position erfasst, per MQTT an einen Raspberry Pi überträgt und live im Browser darstellt.

Der Projektverlauf verlief nicht durchgehend planmäßig. Besonders der Elektronikteil forderte mehr Aufwand als erwartet – vor allem durch Probleme, die erst im Betrieb sichtbar wurden und deren Ursachen systematisch eingegrenzt werden mussten. Die Mechanik überstieg den geplanten Aufwand ebenfalls, da das System bewusst nicht fest verbaut wurde und stattdessen eine lösbare, nachrüstbare Befestigung umgesetzt wurde. Der Informatikteil hingegen ließ sich effizienter umsetzen als geplant, weil sich frühe Architekturentscheidungen – MQTT als Transportschicht, Dash als Dashboard-Framework, SQLite für die Fahraufzeichnung – als tragfähig erwiesen und kaum grundlegende Umbauten nötig waren.

Gegenüber dem ursprünglichen Projektvorschlag hat sich ein wesentlicher Punkt geändert: Die Stromversorgung des ESP32 wurde vom Bordnetz des Motorrads entkoppelt und auf eine externe Powerbank umgestellt. Die RPM-Signalkette ist weiterhin mit der Zündanlage verbunden, aber galvanisch durch einen Optokoppler getrennt. Diese Entscheidung war eine direkte Konsequenz aus Ausfällen durch Spannungsspitzen im Bordnetz und hat die Zuverlässigkeit des Systems deutlich verbessert.

Alle Must-have-Features aus der ursprünglichen Planung sind implementiert und am Fahrzeug in Betrieb. Von den Should-have-Features wurden Streckenaufzeichnung, Kartenanzeige und Fahrthistorie vollständig umgesetzt; Warnmeldungen bei kritischer Öltemperatur sind im Dashboard angelegt. Die Nice-to-have-Features wurden zugunsten von Robustheit und Dokumentationsqualität zurückgestellt.

Abbildungsverzeichnis

Abbildung 1	GPS Sensor	7
Abbildung 2	IMU Sensor	8
Abbildung 3	Temperatur Sensor	8
Abbildung 4	RPM Sensor	9
Abbildung 5	Stromverteiler unter Sitz	12
Abbildung 6	Strom Verteiler	13
Abbildung 7	Case des Sensorboards	13
Abbildung 8	Case des RPM Boards	13

Tabellenverzeichnis

Tabelle 1	Komponentenübersicht	6
Tabelle 2	Zeitsteuerung der loop()-Aufgaben	14
Tabelle 3	rides – eine Zeile pro Fahrt	18
Tabelle 4	trackpoints – ein Messpunkt je 200ms	18
Tabelle 5	Aufwandsvergleich: Planung vs. tatsächlicher Aufwand	26